No silver Bullet

Compara la evolución del hardware con el software.
Bala de plata = soluciones para la crisis, pero ninguna demuestra avances para la escencia solo para el accidente
Esencial: todo aquello que trae aparejado el construir software: conceptos abstractos, estructuras de datos, algoritmos, etc.

Hay 4 aspectos ESCENCIALES que caracterizan la escencia del software

- Complejidad: es inherente debido a la abstracción del SW, crece exponencialmente si crece el SW

- Conformidad: el SW debe realizar aquello que el cliente necesita (adecuarse a requerimientos)

- Mutabilidad: el SW sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo.

- Invisibilidad: al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, como puede ser con estructuras mecánicas. Los grafos son una buena herramienta que permiten modelar muchas situaciones de un software.

Aspectos ACCIDENTALES

- Lenguajes de programación de alto nivel: permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán. Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir.

- Tiempo compartido: reducción de los tiempos de respuesta de un software. Esto permite no desviar el foco de lo que se desea construir.

- Entornos de programación unificados: facilitan la construcción del software al integrar librerías, formato de archivos, frameworks, etc.

Resolución de aspectos accidentales

- Prog orientada a obj
- IA
- Sistemas expertos: sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código,
- Prog automática
- Prog pragmática
- Testing
- Environments and tools
- Estaciones de trabajo poderosas

Resolución de aspectos esenciales

- Comprar antes que construir: Si ya existe = Comprar más barato que construir
- Desarrollo incremental
- Usar prototipos y refinar Req
- Personas capacitadas

Software

Es conocimiento representado en distintos niveles de abstracción, hasta el momento que se incorpora en una computadora.
Es un conjunto de programas con la documentación q lo acompaña y es requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente como así también las herramientas utilizadas para la construcción de este.

Ingeniería de Software

Es una disciplina que se encarga de todos los aspectos de la producción de software, desde la primera etapa "especificación del sistema" hasta el "mantenimiento del sistema" después de que se pone en producción.
Es importante porque:
- Con frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Prevenir errores desde el inicio es mucho más económico que corregir o modificar cosas sobre la marcha cuando el sistema ya se está usando, y esto se logra usando técnicas y métodos de la ing de software

La ing de software se apoya en:
- Se debe a entender el problema y necesidades del cliente antes de desarrollar una solución/software
- El diseño es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño del software, trae 2 beneficios, calidad y facilidad de mantenimiento

Disciplinas que la conforman:
● **Disciplinas técnicas**: actividades enfocadas en construir probar y mantener codigo o artefactos
(Toma y análisis de Req / Diseño, implementación, prueba, despliegue y documentacion de SW / capa usua)

● **Disciplinas de gestión**: actividades de planificación, monitoreo y control del proyecto Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.
permiten que las actividades se adapten a las características del proyecto

● **Disciplinas de soporte**: transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software
(Gestión de Configuración del Software (SCM) / Toma de métricas / Aseguramiento de calidad)
son independientes del marco conceptual y recubren al modelo del proceso (atraviesan del todas las actividades del proceso de desarrollo.)

Crisis de software

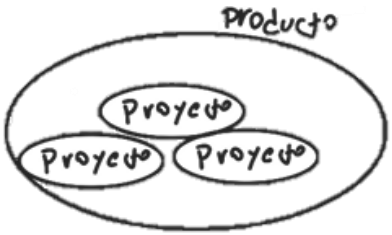
Surge a causa de la evolución de la tecnología del hardware, q permitió la posibilidad del desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software, esto se debe a que el **software es una actividad humana** cuyo **producto es abstracto** e intangible. Tmb lo es la demanda creciente de sistemas complejos, la subestimación de la complejidad, cambios a los que son sometidos los productos para ser adaptados a necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis.

Los sistemas desarrollados que fracasan son principalmente por:
- Demandas crecientes: a medida que la tecnología y técnicas de desarrollo evolucionan, la demanda de sistemas más grandes y complejos aumenta. Los usuarios necesitan que se construyan y distribuyan de manera rápida. La problemática aquí es que los métodos existentes (tradicionales) de ingeniería no permiten lograr esta "espontaneidad", por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
- Bajas expectativas: muchas compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software. Por lo tanto, su software con frecuencia es más costoso y menos confiable. La consecuencia, recae en la conformidad de las empresas y de los usuarios ante un software, es decir, las empresas se conforman con que el software solo "funcione", sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno, desde una perspectiva particular, y definirán flujos de procesos diferentes.
Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra

Ciclo de vida

Serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo, es una representación simplificada de un proceso, la cuál define elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.) y el flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos.
Es decir, son modelos genéricos de los procesos de software; y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente. Y son independientes de los procedimientos de cada actividad del ciclo de vida.

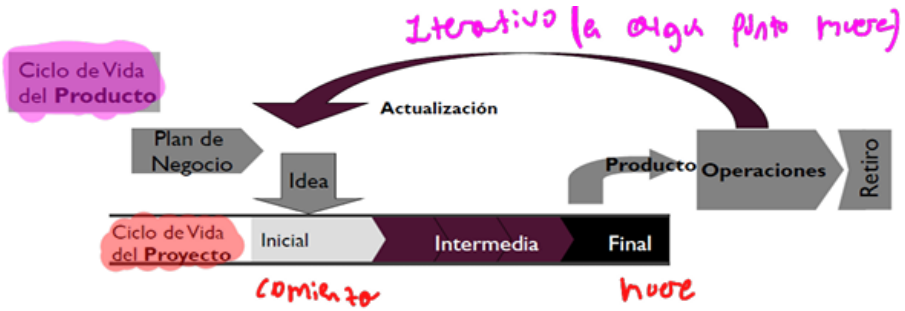


Los modelos de ciclos de vida especifican:
- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar

Un producto puede tener varios proyectos en su ciclo de vida, debido a, cambios o actualizaciones



Clasificación de los ciclos de vida

Modelo Secuencial

Este modelo dispone de actividades de forma lineal, el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada.
El avance entre fases se da tras una revisión al final de cada una de estas para determinar si el software está listo para avanzar.
Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos
Generalmente, este tipo de modelos está dirigido por documentos, el trabajo principal del producto es la documentación del software entre las distintas fases.

Dificultades
- Los proyectos raramente siguen un flujo secuencial.
- Exige que los requerimientos sean completamente explícitos desde un principio (no es habitual)
- El cliente obtiene una versión funcional del producto en etapas muy avanzadas del proyecto
❗ Encontrar un defecto implica un gran rediseño en la solución, ya que está prácticamente todo hecho
- Documentacion excesiva y dependencia entre los equipos de trabajo (B no puede empezó si A no termino)

modelo en cascada puro ayuda a minimizar gastos generales de planificación porque se puede hacer toda la planificación por adelantado. No proporciona resultados tangibles en el desarrollo de software hasta el final del ciclo de vida, es necesaria una definición clara y estable del producto

Modelo Iterativo/Incremental

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software funcional potencialmente entregable. Usualmente los primeros incrementos incluyen las funciones básicas/críticas para el cliente.
Este enfoque vincula las actividades de especificación, desarrollo y validación. El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.
Este ciclo de vida se utiliza en metodologías ágiles

Ventajas frente al modelo secuencial

- Se reduce el costo de adaptar Req cambiantes del cliente y el retrabajo es menor ya que en cada incremento se obtiene una retroalimentación

❗ El cliente es parte del proceso de desarrollo

- La entrega e implementación de SW útil para el cliente es más rápida ganando así, valor del SW

❗ Muy útil para sistemas de requerimientos cambiantes.

Desventajas

- Se invisibiliza el proceso. (es complejo para un cliente saber cuánto avance real hay en un momento dado)
- Alto costo de documentar cada incremento. (versiones ≠ cada 2 semanas = mucha documentacion)
- La estructura del software tiende a degradarse frente a una cantidad considerable de incrementos. (codigo desordenado y frágil culpa de los cambios constantes)

Diferencia entre Iterativo y recursivo: Aunque los dos modelos trabajan en "vueltas" o ciclos, el iterativo/incremental busca ir construyendo el producto por partes, mientras que el recursivo busca resolver riesgos y problemas complejos mediante ciclos sucesivos.

Modelo Recursivo

Es utilizado para **gestionar los riesgos del desarrollo de sistemas complejos a gran escala**. Requiere de la intervención del cliente. El modelo en espiral es un modelo recursivo, que consta de 4 etapas recursivas:
1- Determinar objetivos.
2- Identificar y resolver los riesgos.
3- Desarrollar y probar.
4- Planificar la próxima iteración.

Se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno.
Se inicia con algo en forma completa (puede ser pequeño), se presenta un prototipo que va mejorando con cada vuelta, se generan productos (artefactos o componentes NO HACE REFERENCIA AL PROD DEL PROYECTO) independientes de la implementación, que pueden ser reusables en sistemas de características similares.

Desventajas

- Puede ser más costoso en tiempo y dinero readaptar para reutilizar
- Al reutilizar módulos la tecnología se puede volver obsoleta

PARA SELECCIONAR UN MODELO DE PROCESO

1. requerimientos claros y exhaustivos, contexto de baja incertidumbre y el cliente no requiere obtener el producto rápidamente, es posible aplicar el **modelo secuencial**.

2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces **modelo iterativo- incremental.**

3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el **modelo recursivo**

Proceso de desarrollo de software

Un proceso es una secuencia de pasos ejecutados para un propósito dado.
Mientras que un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos/ARTEFACTOS asociados (documentación, elem de config, codigo, el propio producto, pruebas, modelos).
Los procesos de software son complejos, por lo que no hay un proceso ideal; la mayoría de las organizaciones han diseñado sus propios procesos de desarrollo de software
Para algunos sistemas, como los sistemas críticos, se requiere de un proceso de desarrollo muy estructurado (proceso definido). Para los sistemas empresariales, con requerimientos rápidamente cambiantes, es probable que sea más efectivo un proceso menos formal y flexible (filosofía ágil).

Factores determinantes del proceso

● Procedimientos y Métodos: La adaptación de procesos establecidos para su implementación, dependiendo del contexto, es esencial para la calidad resultante. Definir las actividades que se harán y cuáles no.
Es relevante que esta información esté documentada para lograr transparencia.
● Personas motivadas, capacitadas y con habilidades: Es factor primordial para obtener la calidad del producto. Sin personas motivadas y con capacidades, no se puede lograr construir ningún producto
● Herramientas y Equipos: Materiales necesarios para llevar a cabo el proceso que permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles, lo que mejora la eficiencia de las personas y puedan concentrarse en tareas que requieran de su capacidad.

Actividades fundamentales

- Especificación de software: clientes junto con ingenieros definen el software a producir y sus restricciones.
- Desarrollo: diseño y programación del software.
- Validación: verificación para asegurar que es lo que el cliente quiere.
- Evolución: donde se modifica el SW para reflejar los requerimientos cambiantes del cliente y mercado.

Procesos Definidos

Procesos deterministas: ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.
- Están inspirados en las líneas de producción.
- El objetivo es lograr una repetibilidad mediante administración y control, para poder obtener una previsibilidad de esfuerzo, riesgos, costos, etc. Asociados al desarrollo de software.
- Puedo usar cualquier ciclo de vida.
En la realidad, no es posible obtener el mismo resultado ante mismas entradas ya que depende del contexto, momento y personas

Procesos Empíricos

PILARES: Inspección – Adaptación - Transparencia

Se basan en la experiencia interna y externa de las personas que intervienen en un contexto particular.
NO es considerado determinista ya que no se asume que ante la aplicación del mismo proceso se obtendrán los mismos resultados.
En estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.
Se basan en ciclos cortos de inspección y adaptación (retroalimentación) y mediante un análisis se ajustan de la mejor forma distintos aspectos
La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.
No se pueden combinar con cualquiera de los tipos de ciclos de vida, se suele recomendar que sea el modelo iterativo-incremental, y el que se suele prohibir es el modelo secuencial

Componentes de un Proyecto de Sistemas de Información

Proyecto

Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. **Es una unidad organizativa para adaptar el marco teórico del proceso**, dependiendo de las personas y recursos con los que se cuente
Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

CARACTERISTICAS (Importante porque le dio énfasis en clases)

- **Únicos**: El resultado de un proyecto por más parecido o igual que sea a otro, será único e irrepetible
- **No ambiguos**: deben ser lo suficientemente claros para guiar el desarrollo del proyecto.
- **Medibles**: es necesario medir el objetivo para determinar el avance sobre el proyecto.
- **Alcanzable**: deben ser factibles (que el equipo pueda hacerlos).
- **Duración limitada de tiempo**: Un proyecto tiene un principio y un fin (un plazo).

Cuando se alcanzan los objetivos del proyecto, este llega a su fin.
Un ejemplo de lo que no es un proyecto es una línea de producción, ya que esta se ejecuta indefinidamente en el tiempo.

- **Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos**: Se definen tareas, sus dependencias, recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

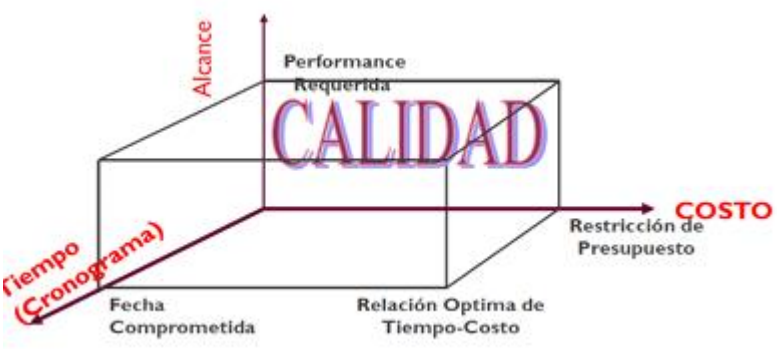
Administración de proyectos de SW (¿????)

La administración de proyectos es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad. La administración de proyectos es una parte fundamental para que el proyecto sea exitoso.
Identifica Req – Establece obj claros y factibles
❗ Una buena gestión **no** puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto
Esta disciplina tiene dos grandes actividades:
- Planificación
- Monitoreo y control.

Objetivo de la administración de proyectos:
- Entregar SW al cliente en el plazo acordado
- Mantener costos dentro del presupuesto
- Entregar SW q cumpla expectativas y entregue valor

La triple restricción – enfoque tradicional

Las variables de restricción de un proyecto son 3: **tiempo, costo** y **alcance**. El balance de estos tres factores afecta directamente la calidad del proyecto (prod de calidad aquel q cumple todo)



Es responsabilidad del Líder de proyecto balancear estas variables.
No es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido.

No se negocia la calidad, si no puedo modificar tiempo y/o costo, debido a que es como entregar un software sin probarlo. La calidad del producto que le entrego al cliente se ve afectada cuando no puedo equilibrar estos 3 factores.

● **Alcance**: Son los Req que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto.
❗Esta variable es la que primero se negocia con el cliente.
En las metodologías tradicionales el alcance es lo primero que se determina y se deja fijo
En las metodologías ágil se va definiendo, refinando y adaptando en base a las retroalimentaciones

● **Tiempo**: Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto al cliente.
En las metodologías ágiles, el tiempo es fijo.
En la metodología tradicional hay una fecha pautada, pero la misma se puede mover en caso de problemas

● **Costo**: Define el costo de todos los recursos implicados en el desarrollo del proyecto
equipamiento, infraestructura, equipos, salarios, entre otros.

◆ El costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto
◆ El alcance es directamente proporcional al tiempo y al costo

Líder de proyecto

En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la opinión del equipo, asigna tareas que deben realizar los integrantes del equipo, y maneja todas las relaciones con los stakeholders del proyecto.
Un líder de proyecto debe contas con las siguientes habilidades:
Poder motivar al personal para que rindan al máximo, adaptar y organizar el proceso existente, fomentar al personal a innovar para ser creativos y crear, ser comunicativo, tener empatía, y saber llevar el liderazgo

Tmb debe contar con habilidades duras (relacionadas a productos, herramientas, técnicas):
Saber resolver problemas identificando conflictos, encontrar una solución o motivar a otros para que la desarrollen
recompensar logros e iniciativas tener influencia sobre el equipo brindar confianza y control

Responsabilidades:
Definir el alcance del proyecto Identificar involucrados, recursos, presupuesto
Detallar tareas, estimar tiempos y Req identificar y evaluar riesgos

Equipo de proyecto

Grupo de personas comprometidas con alcanzar un conjunto de objetivos
- Cuentan con conocimientos, habilidades técnicas, experiencias, personalidades y deben tener un sentimiento de mejora continua y aprendizaje mutuo
- Gralmente son grupos pequeños, permitiendo una comunicación efectiva
- Trabajan de forma conjunta, convirtiéndose en un equipo cohesivo y sinérgico
- Tienen responsabilidad como unidad

Stakeholders

Todos aquellos interesados/involucrados en el proyecto, patrocinador, cliente, equipo, usuarios, etc

Plan de proyecto

Tmb llamado plan de desarrollo de SW, es un artefacto de gestión cumple la función de "Hoja de ruta", en el se documentan las decisiones que se toman a lo largo del desarrollo – división de tareas – asign. de resp. – equipos de trabajo – calendario de desarrollo

En un plan de proyecto se establece qué se va a hacer (alcance del proyecto), cuándo se va a hacer (calendario), cómo se va a hacer (actividades o tareas para cubrir el alcance) y quién lo va a hacer (responsables de cubrir las actividades).

Responder esto implica:

1- Definición del alcance del proyecto

El alcance de un proyecto, junto a su objetivo, responden al qué se está desarrollando.

Es importante diferenciar el alcance de un producto y el del proyecto en sí.

El alcance de un producto define qué funcionalidades debe realizar el SW para cumplir req del cliente

El alcance del proyecto define todo el trabajo y solo el trabajo necesario que se debe realizar para cumplir con el desarrollo de este

2- Definición del proceso y ciclo de vida

Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto.

El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo.

El ciclo de vida define: que trabajo técnico debe realizarse en cada fase – quien debería estar en cada fase – como controlar y aprobar cada fase – como deben ser los entregables – como verificar y validar el prod

3- Estimación

Consiste en predecir el valor de un elemento relacionado al proyecto/producto, el tiempo que va a llevar, costos del sistema, personas necesarias.

En un enfoque tradicional la estimación se realiza una sola vez por el líder de proyecto y existe un orden recomendando para estimar:

a) Estimar el **tamaño del prod** y para esto son necesarios los Req, esta estimación es la más importante ya que afecta al resto de aspecto

b) **Esfuerzo** es la cantidad de horas personas lineales haciendo una única actividad por vez dentro del desarrollo, esta estimación intenta responder cuantas horas son necesarias para construir el producto (el esfuerzo repercute mucho en el costo del proyecto, casi 80%)

c) **Calendario**, permite responder cuando se van a realizar las tareas definidas, en el momento de la estimación no es necesario realizar una calendarización especifica a nivel día si no solo escalara las tareas a realizar

d) **Costos** depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del SW, esto es lo último que se estima

4 – gestión de riesgos

Un riesgo es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.
Es importante identificar y definir los riesgos asociados a un proyecto para analizar y definir cual atender

Tipos de riegos:

- Riesgo del proyecto: aquellos que alteran el calendario o los recursos del proyecto
(renuncia de un diseñador experimentado)

- Riesgo del producto: aquellos que afectan la calidad o el rendimiento del software a desarrollar
(El software no cumple un requisito importante)

- Riesgo organizacionales: riesgos que provienen de la org donde se desarrolla/implementa el proyecto
(Se cancela el presupuesto)

Para medir riesgo se utiliza la "exposición al riesgo" ER = P(RI) * I(RI)
expo al riesgo = probabilidad de resultado insatisfactorio * perdidas de las partes afectadas por un R insatisf
Se obtiene como el producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera. Este valor permite organizar y gestionar los riesgos, y se debe hacer foco en reducir la exposición del riesgo.

Antes los riesgos, es posible tomar 3 actitudes:
• Negación: hacer de cuenta que los riesgos no existen.
• Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
• Gestión Proactiva: ir trabajando sobre el riesgo generado planes de mitigación y de contingencia.

Grandes riesgos: Volatilidad de Req / Abandono de integrantes / Falta de capacidad del equipo

GESTION DE RIESGOS

1. Identificación de riesgos: se identifican los riesgos que suponen una mayor amenaza al SW u Org.
2. Análisis de riesgos: P/c riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto
3. Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos.
consisten en considerar acciones a tomar para minimizar o evitar el impacto que representa el riesgo
4. Monitorización del riesgo y control: permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado.



5 - Asignación de responsabilidades (o recursos)

La asignación de responsabilidades a personas permite asignar roles a los integrantes del equipo de trabajo
Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol

6 - Programación de proyectos (calendarización)

En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo. Se toma como base lo definido en la estimación del calendario realizada previamente pero se refina al máximo detalle posible cada actividad (descomponer el proceso de desarrollo en actividades identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar)

7 - Definición de métricas

Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio
Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.
Las métricas se pueden clasificar en: Tamaño del producto - Esfuerzo - Calendario (tiempo) - Defectos – Costos

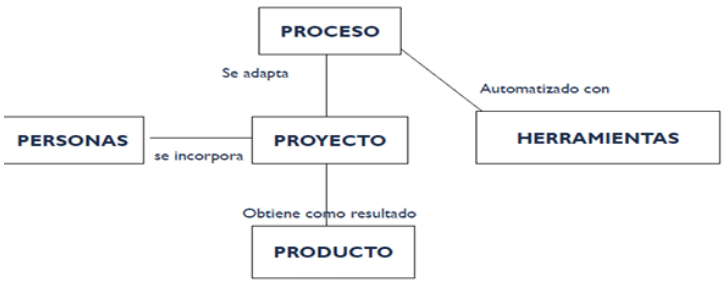
8 - Definición de controles

En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

¿Planes de Soporte?
Planes de Testing / mantenimiento / subcontrataciones / calidad / capacitaciones / iteraciones

Causas de fracasos de proyectos
- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo.

proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.



El **proceso**, automatizado con **herramientas**, se adapta al **proyecto**, al cual se incorporan **personas**, y de este se obtiene como resultado un **producto**

Previo a la definición del proyecto, es necesario determinar:
- Objetivos (permiten identificar metas globales del prod sin especificar como se lograra)
- Ámbito del producto (define funciones y comportamientos que caracterizan al producto)

Proceso: proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software. Define como se va a desarrollar el software, y el conjunto de actividades.
Personas: parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva. Si se adopta el “mejor proceso” y se destina mucho esfuerzo a planificar, de nada servirá si no tenemos un equipo capacitado que reúna las habilidades y herramientas necesarias

Pilares del empirismo

Transparencia: La información importante debe ser visible y entendible para todos.
Transformar el conocimiento de cada uno en conocimiento explícito para todos
Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo.
La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

Inspección: Se observa y revisa frecuentemente lo que se está haciendo para detectar problemas.
La inspección puede realizarse para el producto, procesos, personas, etc. Se realiza por todo el equipo y permite detectar varianzas o problemas
La inspección permite la adaptación. La inspección sin la adaptación se considera inútil.

Adaptación: Si la inspección muestra que algo no funciona, se modifica el rumbo, es la mejora continua y capacidad de adaptarse en función de los resultados de la inspección, se realizan ajustes (lo antes posibles para minimizar la desviación)

Filosofía Agil

Metodología = MALA PALABRA ==> decir enfoque o conj de principios

Explicación de FranelK porque el resto es una poronga:
La **filosofía agil** es un movimiento/ideología que define un conjunto de enunciados y pautas para el desarrollo de un producto (¿de SW?), establece que el obj principal es entregar SW de manera frecuente en un entorno cambiante = **Req cambiantes**, y es necesario involucrar al cliente para una retroalimentación rápida sobre dichos Req.
Se adopta el **ciclo de vida Iterativo incremental** y se hace uso de **procesos empíricos.**
Se sostiene que el SW no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema
(Los incrementos se hacen de 1 a 4 semanas – se entregan – se recibe una retroalimentación permitiendo la **Inspección y adaptación** sobre los Req cambiantes)
mientras que la "materialización" de esta ideología es el: **Manifiesto Agil** es un documento concreto en donde se estableces los 4 valores y los 12 principios

Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.

¿Cuándo es aplicable agil?

La ideología Agil es aplicable en contexto complejos (entendimiento y certeza parcial)
Existen distintos "Frameworks agiles" que son orientados a la gestión de proyecto y no al desarrollo

Aplicar la ideología agil permite: - Acelera entregas de productos de valor para el cliente
- Adaptarse fácilmente a cambios
- Alinear los objetivos organizacionales y de TI
- Reduce riesgos de proyecto
- Reduce costos de proyecto
- Mejor visibilidad del proyecto

Valores Agiles

El Manifiesto Ágil, expresa valores, los cuales priorizan ciertos aspectos por sobre otros, pero sin restar importancia a ninguno de ellos

1- Individuos en interacciones, por sobre procesos y herramientas:

Es preferible un equipo de personas motivadas con un buen funcionamiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos.

El desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo (como así también paliar las debilidades individuales), en lugar de intentar homogeneizar un equipo, para así lograr una armonía que permita fortalecer la unión y motivación dentro del equipo. Los procesos y herramientas deben ser un complemento para mejorar la eficiencia y éstos se deben adaptar al equipo y no al revés.

2- Software funcionando por sobre documentación extensiva:

Priorizar el software asegura que se tengan versiones estables, incrementales y mejoradas del SW al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando en aquellas funcionalidades de mayor valor para el cliente.

Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (que le sea útil a aquel que va a utilizar el SW)

El enfoque ágil lo que plantea es que se debe generar la documentación cuándo sea necesario. No plantea "no generar documentación".

3- Colaboración con el cliente por sobre negociación contractual (contrato rígido)

Permitirá tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente y que el equipo agil se integre con los stakeholders.

Esto reduce la dependencia de documentación extensiva ya que cada miembro del equipo participa en la toma de decisiones que solían requerir comunicación escrita

Implica comprometer al cliente como parte importante del producto, se trata de tener a un cliente que esté comprometido con cada entrega, al momento de probarlo y dar el feedback.

A veces las negociaciones contractuales imponen restricciones que impactan de manera negativa en las partes, se traduce en un mayor distanciamiento en la relación equipo de desarrollo-cliente. A pesar de que los contratos sean necesarios, sus términos y condiciones, dificultan el ida y vuelta

El problema de las negociaciones contractuales en las empresas empiezan cuando el cliente tiene ya una especie de acuerdo formal (contrato firmado) con el equipo por un monto y un plazo y una cantidad de características del producto de software que hay que entregar y el cliente viene y dice "me olvidé de esto" o "necesito esto" y ahí es donde empiezan los puntos de fricción por qué es donde comienzan las discusiones porque ya existe un contrato firmado donde se establecieron todas las pautas.

4- Responder a cambios por sobre seguir un plan:

Esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, con el paso del tiempo surgirán nuevas necesidades.

Esto está ligado al entorno cambiante, por lo que un equipo ágil está abierto a los cambios, para responder ante ellos de una manera rápida y eficiente entregando el mayor valor posible, en lugar de apegarse y seguir un plan definido confeccionado en etapas tempranas de requerimientos.

Los principales valores (Principios ≠ valores) de la gestión ágil son la **inspección y la adaptación**, diferentes a los de la gestión de proyectos tradicional: **planificación y control** que evite desviaciones del plan

Principios Agiles

1- Nuestra mayor prioridad es satisfacer al cliente
Un equipo agil desarrolla y entrega funcionalidades en el orden (importancia) que defina el PO para que cada incremento dé el mayor valor posible (se utilizan user stories para definir un valor significativo y asegurar que el equipo responde a las necesidades)

2- Aceptar que los requisitos cambien
Incluso en etapas tardías de desarrollo se plantea que el equipo ágil está abierto al cambio, al no seguir un plan estructurado, permite que el equipo sea flexible, y responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.

3- Entregar software funcional frecuentemente
es imprescindible que un equipo ágil al finalizar cada iteración haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable.
Es importante que ese código pueda ser desplegado (que cumpla con la DoD)

4- Técnicos y no técnicos trabajando juntos durante todo el proyecto
El PO que esta del lado del cliente es importante que participe con el equipo, ya que el PO debe representar a los interesados expresar sus necesidades y tomar decisiones sobre el producto
El PO guía las iteraciones, escribe US y gralmente no cuenta con la disponibilidad requerida por el enfoque agil, siendo un talón de Aquiles en el proceso

5- Desarrollamos proyectos en torno a individuos motivados
Los participantes de un proyecto deben estar motivados y comprometidos con el obj del mismo, deben verse entre si como un único equipo, colaborar entre si y no limitar sus tareas y delegar

6- El método más eficiente de comunicar información es con conversaciones cara a cara
lo que dice el titulo pa, asi de facil reuniones presenciales >>>>> reuniones virtuales / msj

7- La mejor métrica de progreso es el software funcionando
El manifiesto ágil plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente
Metodología Tradicional: tienen muchas métricas y docu sobre como medir el proyecto, pero de poco sirve si el producto no refleja valor al cliente

8- Los procesos ágiles promueven el desarrollo sostenible
La filosofía ágil promueve la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones, dinámica de trabajo y madurez (hoy en día mucha migración de empleados)

9- La atención continua a la excelencia técnica y al buen diseño mejora la agilidad
La calidad del producto es un aspecto NO negociable
Hay que atender las necesidades principales del cliente y entregar el codigo con la mejor calidad posible

10- La simplicidad es esencial
Como equipo hay que enfocarse en necesidades que REALMENTE APORTEN VALOR al cliente y no perder el tiempo en detalles que no satisfacen Req – ni hacer cosas que el cliente no pidio

11- Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados
Los equipos de desarrollo agiles donde no hay jefes que impartan ordenes, tienden a ser mas proactivos, aportando soluciones, conceptos, etc útil para el equipo y toman decisionmes cuando creen que es adecuado

12- A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar
Se basa en 2 de los pilares empíricos (Inspección y adaptación)
En los equipos cada determinados períodos de tiempo se evalúan distintos aspectos: Desempeño – variación de personal – tecn que no funciono – riesgos. Y con esto se reflexiona sobre oportunidades de mejora.
Esto permite adaptar aspectos necesarios, corregir o mejorar

Filosofía LEAN

Filosofía / enfoque más viejo que agile, busca maximizar el valor generado al cliente con el mínimo consumo de recursos, sostiene que todo esfuerzo que no cree valor es desperdicio (reduciendo asi el esfuerzo en crear artefactos inútiles)
busca atender al cliente con alta calidad – bajo costo – bajo tiempo de entrega y satisfaciendo las necesidades del cliente
Utiliza el método just in time

Principios lean

1- Eliminar desperdicios

El proceso debe ser eficiente, debe sumar valor en todos los pasos y todo paso o parte que no contribuye a la generación de valor produce un desperdicio, busca evitar: funcionali. en exceso que no se usen (desperdicio) retrabajo – artefactos obsoletos
DESPERDICIOS LEAN:
- Características extras que no tienen que ver con las funcionalidades
- Trabajo a medias – sin terminar, teniendo que ser retomado a posterior
- Proceso extra (pasos extras que se ejecutan en el proceso de desarrollo que no aportan valor al producto)
- Búsqueda de información
- Defectos, errores que se trasladan entre etapas, provocando retrabajo
- Esperas – tiempo de espera por lo que entendí seria como que una tarea depende de q otra sea terminada
- Cambios de tareas (Multitasking) genera pérdida de tiempo en prepararnos para realizar cada tarea (seteo)

2- Amplificar el aprendizaje

Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo.

3- Embeber la integridad conceptual

La calidad del producto no se negocia y la simplicidad es un factor clave, el producto tiene que satisfacer las expectativas y necesidades del cliente (prioridad 1) no se debe perder de vista que esa es la prioridad principal. Para esto se deben encastrar todas las partes del producto que tengan coherencia y consistencia para hacer al producto más íntegro

4- Diferir compromisos hasta el último momento responsable

Postergar la toma de decisiones lo suficientemente como para poder tener la mayor información necesaria para tomar esa decisión, pero a la vez, antes de que ese momento sea muy tarde

5- Dar poder al equipo

Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados
Esto implica no subordinar por parte de los superiores, debido a que esto anula las capacidades intelectuales
Pero a la vez, implica un compromiso por parte de los miembros del equipo, para desempeñarse y cumplir con lo pactado de forma completa y correcta.

6- Ver el todo

Tener una visión holística, que permita asociar y comprender el todo, el producto, el valor agregado que hay detrás, más allá de los objetivos particulares por áreas o gerencias, porque la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.

7- Entregar lo antes posible

La idea es entregar al cliente SW funcionando lo más rápido posible y que sea útil para él. Se entrega software antes de que las necesidades de negocio cambien, porque el ambiente donde se desempeña el mismo cambia. Para esto Lean propone acotar ciclos de desarrollo y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo que sea valioso

Relación Lean-Ágil

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados, de forma tal que la sinergia de este facilite el desarrollo del proyecto. Esta sinergia, se interpreta como un valor agregado para el cliente. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es también un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua.

Fundamentación de los requerimientos ágiles

La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el mismo no está completamente definido.

Los pilares de los Req agiles son:

- Usar el valor para construir el producto correcto:

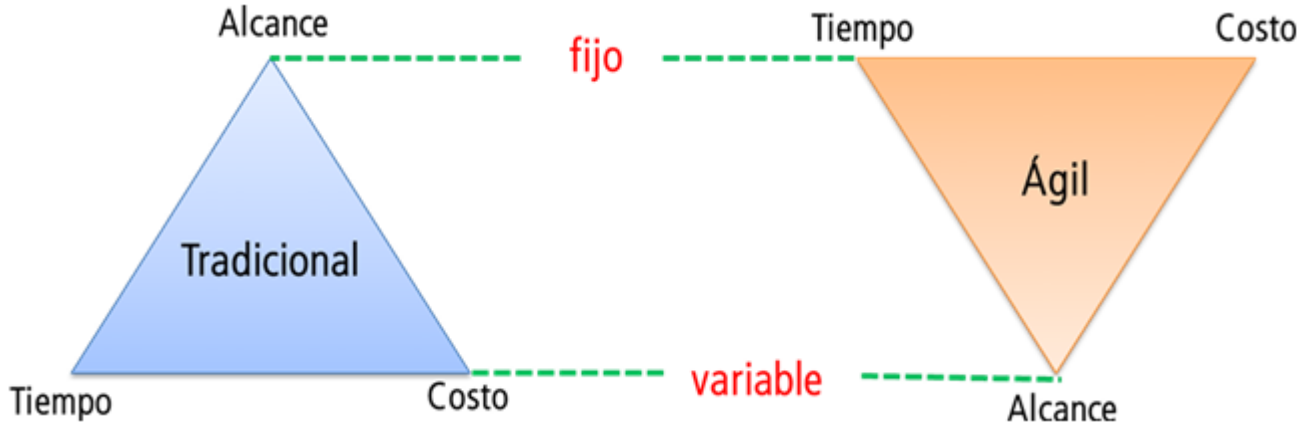
Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio.

- Determinar que es "sólo lo suficiente":

Los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso, se debe buscar lo mínimo y NECESARIO

- Usar historias y modelos para mostrar que construir

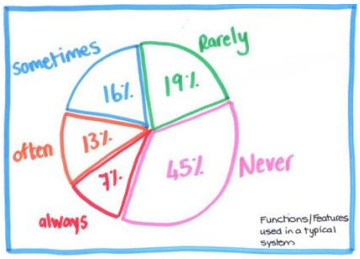
La triple restricción – enfoque ágil



Tradicional = dirigido por un plan
Ágil = dirigido por valor

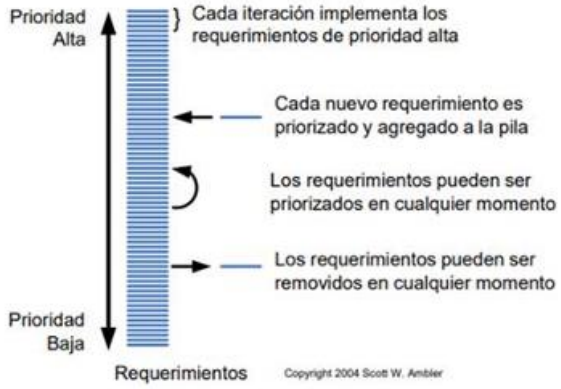
BRUF – Big Requirements Up Front

BRUF es una forma de trabajar en la que se intenta definir y documentar la mayor cantidad posible de requisitos del sistema antes de comenzar a desarrollarlo.
Básicamente definir "todo" lo que necesita el cliente y los requisitos del sistema, lo cual trae PROBLEMAS, ya que es imposible conocer todos los requisitos desde el comienzo de un proyecto. El cliente o los usuarios pueden no tener completamente claro qué necesitan, a medida que se va utilizando el sistema van surgiendo nuevas necesidades



Una de las razones por las cuales necesitamos cambiar el enfoque, y determinar solo lo que suficiente, es porque todos los productos de software tienen un desperdicio significativo, de todas las funcionalidades desarrolladas, es muy alto el porcentaje de aquellas que el cliente/usuario no usa.

Product Backlog



La gestión agil responde a la mala practica de BRUF mediante el Product backlog.
El cual es una lista de características y Req del software ¿donde cada item esta ponderado / con mayor o menor prioridad?
El PO define toda esta lista, ya que es el q tiene claro las necesidades y Req principales (además obvio de asignar que Req deben ser atendidos primero)

Req importantes ↑ Req menos importantes↓

Tradicional = cliente no se compromete **agil** = el cliente forma parte del equipo

Requerimientos Just in Time

Analizo cuando lo necesito, no antes ni después (que no se haga tarde).
Analizo con detalle un requerimiento cuando tenga el suficiente valor

Es un concepto que se usa fundamentalmente para evitar desperdicios (que en este contexto sería invertir tiempo en especificar requerimientos que después cambian).
El producto no va a estar especificado 100% desde el principio, vamos a ir encontrando requerimientos y describiéndolos conforme haga falta. Lo que nosotros tenemos que entregarle al cliente es valor de negocio, no características de software. El software es el medio por el cuál nosotros le entregamos valor de negocio.

METODOLOGIA TRADICIONAL vs METODLOGÍA ÁGIL

	TRADICIONAL	ÁGIL
Prioridad	Cumplir el plan	Entregar Valor
Enfoque	Ejecución ¿Cómo?	Estrategia (¿Por qué? ¿Para qué?)
Definición	Detallados y cerrados. Descubrimientos al inicio	Esbozados y evolutivos. Descubrimiento progresivo
Participación	Sponsors, stakeholder de mayor poder e interés.	Colaborativo con stakeholders de mayor interés (clientes, usuarios finales)
Equipo	Analista de negocios, Project Manager y Áreas de Proceso.	Equipo multidisciplinario.
Herramientas	Entrevistas, observación y formularios	Principalmente prototipado. Técnicas de facilitación para descubrir.
Documentación	Alto nivel de detalle. Matriz de Rastreabilidad para los requerimientos.	Historias de Usuario. Mapeo de historias (Story Mapping)
Productos	Definidos en alcance	Identificados progresivamente
Procesos	Estables, adversos al cambio.	Incertidumbre, abierto al cambio

Fuente de requerimientos

En la gestión agil se promueve que los requerimientos del software se obtienen conversando cara a cara con el cliente.
En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo (sería lo que en el gráfico dice "papel").
Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Esto permite reducir la cantidad de documentación formal.
El desarrollo ágil no implica la no documentación o la no especificación de requerimientos, sino que prioriza documentar y especificar aquellos que sean de valor para el cliente, antes que hacer toda una ERS errónea.

Momento de la captura de requerimientos
En el enfoque tradicional, los requerimientos son definidos al principio del proyecto, el flujo de trabajo de requerimientos tiene un gran peso en la fase inicial y de elaboración.
En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto. El inicio de la construcción del software se da en un contexto en el cual los requerimientos no están completamente definidos, sin embargo, se tiene una visión.

Contenedor y persistencia de los requerimientos
El enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS.
En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de requerimientos, ya que una vez implementados estos se "persisten" en el producto funcionando.

Tipos de requerimientos



La parte baja de la pirámide, corresponde a los Req del Dominio de la solución no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable.
Lo que está por encima, es Req de Usuario y Req de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

Req negocio → Req usuario → Req funcional → Req no funcional → Req de implementación

- Las **User Stories** son requerimientos de usuario alineados a un requerimiento de negocio.
  - no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.

- Al usuario le interesa el requerimiento de negocio (valor de negocio), pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.

- El foco de la gestión ágil de requerimientos está puesto sobre el usuario y sobre sus necesidades

DE NEGOCIO	Son los de más alto nivel, nos referimos a cuales son los requerimientos en términos de la visión del negocio
DE USUARIO	Tienen que ver concretamente con los requerimientos de usuario final
FUNCIONALES	Podríamos relacionarlo 1 a 1 con los casos de uso en la gestión tradicional
NO FUNCIONALES	Suelen ser los más ocultos, que el cliente no tiene en claro y debo especificarlos. Un requerimiento no funcional puede hacer que el software que estoy desarrollando no sirva (IMPORTANCIA FUNDAMENTAL)
DE IMPLEMENTACIÓN	Se suele decir que están dentro de los no funcionales, pero tienen que ver con cuestiones de restricción o implementación específicos.

Debemos Asumir/tener en cuenta que:
- Los cambios van a existir todo el tiempo de forma constante
- Debemos tener una mirada de todos los que están involucrados: todos los que tienen algo para decir del producto (stakeholders).
- "El usuario dice lo que quiere cuando recibe lo que pidió". es por esto que las iteraciones cortas y las entregas continuas son esenciales, ya que permiten una retroalimentación y un enriquecimiento para futuras entregas
- No hay técnicas ni herramientas que sirvan para todos los casos. debemos detectar para cada situación cuál es la herramienta más adecuada.
- Lo importante no es entregar una salida, un requerimiento, sino es entregar un resultado, una solución de valor



User Stories

Lo mas difícil del SW es decidir que queremos construir y como va a satisfacer las expectativas del cliente, un problema que provoca construir algo que no necesita/quiere el cliente es no asimilar la complejidad que implica hacer SW y que se subestima el relevamiento de Req
Tmb suele haber problemas de comunicación entre clientes y equipo:
- Cuando solo el cliente domina la comunicación pide muchas funcionalidades y fechas de entrega que los dev no pueden cumplir
- Cuando solo el equipo domina la comunicación la jerga técnica reemplaza el lenguaje y se pierde la oportunidad de aprender que se necesita escuchando al cliente
- Lo óptimo es una buena comunicación de ambas partes

Como usuarios ver entregas tempranas permite que surjan ideas nuevas o ver que si es o no lo que queríamos antes de que el proyecto siga avanzando

ENTONCES, las US son una, técnica para trabajar requerimientos de usuario en ambientes ágiles contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.
- Las US son escritas por el Product Owner.
Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.
Otros propósitos de las US son: Representan una necesidad de usuario - Conforman una descripción del producto - Son utilizadas como ítems de planificación
❗ La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas
La elicitación tradicional no da buenos resultados, ya que todas las decisiones se toman al principio del proyecto cuando menos información hay. Por lo que se estableció que esta toma de decisiones se traslada a lo largo del proyecto, a medida que la incertidumbre baja y la información sube, buscando que se obtenga más a menor tiempo posible

Partes de la User Story
(3C = card/conversación/confirmación)

1- Tarjeta
Descripción de la historia

<<Frase Verbal>>
Como <<**nombre del rol**>> yo puedo <<**actividad**>> de forma tal que <<**valor de negocio**>>

- Nombre del rol: representa quien está realizando la acción / recibe el valor
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US.
- Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la US.
- Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad.
- Pruebas de aceptación: POR LO QUE YO ENTENDI, son básicamente pruebas de implementación que determinan si una US esta completa (por lo que vimos en practico están relacionados a los criterios de aceptación)

2- Conversación
Parte más importante de la US, explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US.

3- Confirmación
Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Ventajas	Desventajas
Prioriza la comunicación verbal (importante) por sobre la escrita	En proyectos grandes, es difícil entender la relación entre U.S (son independientes entre sí).
Son comprensibles tanto para el cliente (PO) como para el equipo de desarrollo	No son apropiadas para equipos muy grandes (por el tema de priorizar la conversación verbal, por sobre la escrita).
Tienen el tamaño justo para permitir planificar	
Permiten el desarrollo iterativo	
Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios	

Product Backlog y las User Stories

El Product backlog es una pila de PBIs - Product Backlog Ítems, cada una con su estimación
En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (granularidad fina)
En la parte más baja de la pila, se ubican las U.S. de menor prioridad (granularidad gruesa, no indispens)
En cualquier momento se puede cambiar la priorización, según cómo lo indique el PO

Para quemar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad) se quitarán para implementarlas en la próxima iteración.

Porciones verticales

Las US son porciones verticales, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.
Debemos diseñar el pedazo de C/cosa que nos hace falta para que una determinada característica funcione.

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.

Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.
El tamaño de las U.S. se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una U.S: Importante para la estimacion

Complejidad: hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas.
Por ejemplo, algoritmos complejos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.

Esfuerzo: es el tiempo en horas de trabajo que requerirá la implementación.

Incertidumbre:
○ Técnica: cuando se tiene dudas en el dominio de la solución.
Por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
○ De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Points

Son unidades de estimacion - medida relativas (**no importa el valor en sí, sino su comparación**), las cuales miden el tamaño de un producto, NO una medida basada en el tiempo (por ejemplo, horas hombre). Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores.
"relativo" surge a partir de que se comprobó que las personas no saben estimar en términos absolutos, sino que les resulta útil la comparación de tamaños.

Niveles de abstracción

1. USER STORY: cumple con el criterio de Listo (DoR) para ingresar a una iteración.
2. ÉPICAS son historias de usuario muy grandes, que en principio son así de grandes porque están en un lugar de la pila en donde todavía no fueron detalladas. Como todavía no debo implementarlas las dejo plasmadas como una gran US sabiendo que en algún momento voy a tener que llegar a disolverla, detallarla y hacerla más pequeña.
3. TEMAS, que incluso podrían ser hasta más grandes que una épica. Es un conjunto de US relacionadas que defino en función de un título para recordarme que todo lo que está incluido en ese tema lo voy a tener que tratar en algún momento
4. SPIKE (tipo especial de US) sirven para quitar riesgo o incertidumbre de otro requerimiento, de otra US o de alguna faceta del proyecto que nosotros queremos investigar. Son específicamente creadas para esto. Ejemplo: investigar una tecnología que no conocemos, dudas de cómo plantear alguna interacción del usuario con el sistema.
- SPIKES TÉCNICAS son utilizadas para investigar enfoques técnicos en el dominio de la solución. Cualquier situación en la que el equipo necesite una comprensión más fiable sobre alguna tecnología a aplicar antes de comprometerse a desarrollar una nueva funcionalidad en un tiempo fijo.
- SPIKES FUNCIONALES son utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Modelado de Roles

Se utilizan tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario.
El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad.
Por esto existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales

1. **Personas**: esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
2. **Personajes extremos**: que personas en términos extremos podrían llegar a utilizar el software.
3. **Proxies** (Usuarios Representantes): es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de aceptación

Define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, funciona como parámetro para que los desarrolladores determinen si la misma fue cumplimentada o no.
Lo fundamental es que esté escrito en términos objetivos y concretos.
Es información concreta para saber si lo que implementamos es correcto o no.
● Definen límites para una User Story (US)
● Ayudan a que los PO respondan lo que necesitan para que la US provea valor (Req funcionales mínimos).
● Ayudan a que el equipo tenga una visión compartida de la US.
● Ayudan a desarrolladores y testers a derivar las pruebas.
● Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

Criterios de aceptación buenos

• Definen una intención, no una solución. {El usuario debe elegir al menos una cuenta para operar}
• Son independientes de la implementación.
• Relativamente de alto nivel, no es necesario que se escriba cada detalle.

Los detalles como - El encabezado de la columna se nombra “Saldo” - El formato del saldo es 999.999,99
Estos detalles, resultado de las conversaciones con el PO y el equipo, **pueden capturarse en dos lugares**:
- Documentación interna de los equipos
- Pruebas de aceptación automatizadas.

Pruebas de aceptación

Son declaraciones de intención de que hay que probar.
El conocimiento del negocio está inmerso en las pruebas de aceptación.
❗ Se prueban escenarios exitosos y escenarios que fallen.
En las pruebas de aceptación se encuentran las más importantes, no es posible escribir todas.
El PO acepta la User como implementada si todas las pruebas de aceptación se cumplen.
❗ Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores
❗ Las mismas deben escribirse antes que la programación empiece, y se recomienda que las escriba el cliente.

❗DoR (Definition of Ready) – Definición de Listo❗

Es una **medida de calidad** que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model.
Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

❗INVEST Model❗

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

• **Independiente**: la User es calendarizable e implementable en cualquier orden.
No existe una dependencia con otra User para desarrollarla o para mostrarla al Product Owner.
El PO tiene la libertad de priorizarlas en el Product Backlog como él desee.
{dato de fnk: hay algunos datos/info que se puede mockear y ahí deja de ser dependiente de otra US}

• **Negociable**: Las US no son contratos estrictos, sino descripciones de funcionalidades donde se describen las necesidades de los usuarios, y los detalles se dejan para negociar en conversaciones futuras entre clientes y el equipo de desarrollo.
Esto implica que no deben detallarse todos los aspectos de forma exhaustiva, sino más bien, servir a modo de recordatorio para negociar aquellos aspectos en un futuro cuando se deba implementar.
El cliente debe expresar lo que necesita, la US está centrada en el "qué" no el "cómo". El cómo es decisión del equipo y debe ser negociable con el PO.

• **Valuable**: Una US debe ser valorable para los clientes. Es decir, las U.S. deben aportar valor de negocio a quiénes está destinado el producto, no para el equipo de desarrollo.

• **Estimable**: la US debe contener la cantidad de información suficiente para estimar el tamaño, complejidad y esfuerzo necesario para realizarlo.
Si no es posible estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida (No puedo saber cuánto tardaría). Hay tres razones por las cuales una US puede no ser estimable:
- El equipo tiene incertidumbre acerca del dominio (hablar con el PO)
- El equipo tiene incertidumbre técnica, es decir, sobre el dominio de la solución (implementar Spikes)
- La US es demasiado grande (dividir en US mas chicas)

• **Small (Pequeña)**: la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración.
Se debe comenzar y terminar una característica en una iteración. El tamaño es relativo. ya que depende del equipo, de la experiencia, de su capacidad de trabajo y de la duración de la iteración. Ágil aplica gestión binaria, es decir, si se puede completar la US en la iteración, es pequeña. No aplica porcentaje de avances.

• **Testeable**: la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

DoD (Definition of Done) – Definición de Hecho

- Determina si la US está terminada.
Son criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al PO, para que decida si se pasa a producción o no.
En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.
Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva del trabajo qué se realizó, para lograr ese incremento.
En la práctica consta de un Checklist de condiciones que deben cumplirse. Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una Definición de Hecho adecuada para el producto.

Temas de Inversión

Representan un conjunto de iniciativas o propuestas de valor que conducen la inversión de la empresa en sistemas, productos, aplicaciones y servicios con el objetivo de lograr una diferenciación en el mercado y/o ventajas competitivas.

1. Inversión en productos existentes, mejoras, soporte y mantenimiento.
2. Inversión en nuevos productos y servicios que mejorarán los beneficios
3. Inversión a futuro en productos y servicios avanzados.
4. Inversión hoy, pero que no dará beneficios hasta dentro de unos años.
5. Inversión en reducción (estrategia de retiro) para ofertas existentes que están cerca del final de su vida útil.

Estimacion

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto.

La estimación es una Predicción analítica e imparcial del esfuerzo o costo
La objetividad viene porque se utilizan ciertos parámetros para poder brindar una probabilidad de ocurrencia y no certeza
Estimar es predecir un hecho que va a ocurrir en un futuro
- Estimar tiene asociado un factor de probabilidad
    -Generalmente se confunde con un acuerdo/compromiso
    - Debería estar acompañado con el % de probabilidad de que dicha estimación se cumpla

No ser dogmático sobre nada (no ver como un caballo solo para delante, ver al rededor y entender en que momentos estas y cuál es el contexto), ser pragmático es la clave del éxito en casi todo
No se pueden tomar decisiones sin tener contexto

Compromiso:
Una promesa de entrega asumiendo riesgos específicos, **se acuerda dsp con el cliente una vez que ya se realizó una estimación**
La planificación toma como entrada una estimación y logra basándose en ella establecer con el cliente este compromiso, el cual puede ir cambiando a medida que vas desarrollando el proyecto

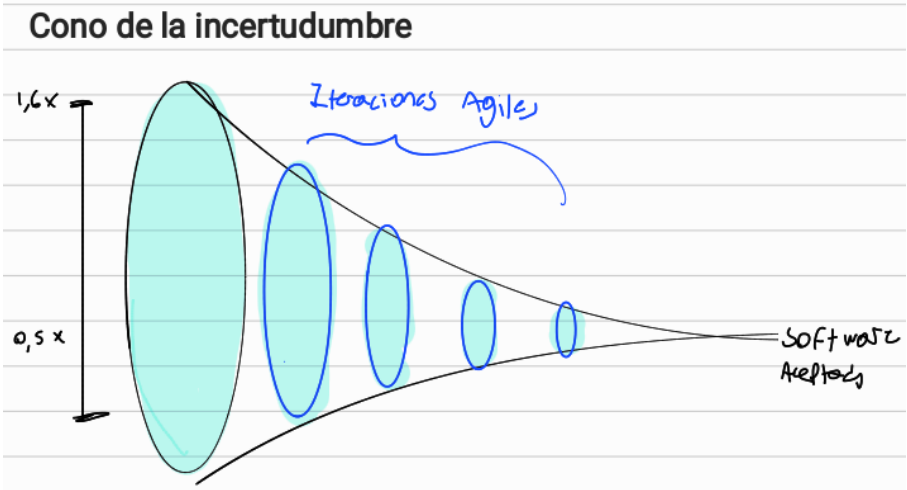
Lo que normalmente pasa en la dinámica de los proyecto de desear de SW
1- Se parte de un a estimación con compromiso, puede ocurrir que el ¿Req? técnico que se llevó a cabo no sirva y te manden a hacer todo de nuevo
Cronogramas subestimados (no son realistas porque nos fuerzas a plantear algo ajustado a las capacidades del ¿equipo?)

2- Al estar subestimado se recortan actividades por falta de tiempo

3- Se programan soluciones rápidas y sucias diciendo "Dsp lo arreglamos" esos arreglos nunca llegan y queda un codigo feo

4- Como consecuencias surgen altas tasas de defectos, con suerte en la etapa de testing y lo peor en producción

5- Como consecuencia de los errores surgen las reuniones/comité de crisis donde todo el mundo se altera



las estimaciones se piden demasiado pronto a medida que se cierra el cono la falta de info va disminuyendo

BUENA ESTIMACION
- Una buena estimacion tiene que estar representada en rangos no una fecha en específico, porque eso genera una probabilidad gigante de no acertarle la fecha (Gralmente un 20% de desviación para arriba y para abajo)

- Incluye un porcentaje de probabilidad explicito
80% de que se cumpla
75% de confianza

- Esta totalmente separa del objetivo del negocio

- Habilita decisiones de control, todo este proceso debe permitir comprometerse en términos de ser confiables

Errores en las estimaciones
- Información imprecisa acerca del software a estimar o de la capacidad para realizar el proyecto.
- Proyecto mal definido
- Omitir aspectos necesarios para la estimacion del proyecto como, Req, licencias, reuniones, revisiones
❗ No tener presente que planificar ≠ estimar, las estimaciones son la base de los planes y los planes no tienen que ser lo mismo q lo estimado
- A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo
- Las estimaciones **no son compromisos**

¿Por qué estimamos?

Existen dos razones principales por las cuales se llevan adelante las estimaciones:
- Para predecir completitud
- Para administrar riesgos

Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final.
Las estimaciones requieren:
- Experiencia
- Acceso a buenas métricas
- Valor para comprometerse con predicciones cuantitativas cuando la info cualitativa es todo lo que existe.

Siempre se quiere saber muy al inicio del proyecto cuál será el costo, tiempo que llevará, etc, y como aún no se conoce con precisión cuál será el alcance del producto resulta complejo predecir con exactitud estos aspectos, por eso a medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con mayor información sobre la cual realizar las estimaciones.
(Se suele decir que típicamente la primera estimación difiere hasta un 400%).

Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a dos aspectos:
1. Al inicio del proyecto tenemos poca información y demasiada incertidumbre = aumenta el riesgo de generar estimaciones erróneas.
2. Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
3. “el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces”.

Para estimar:

Primero se define el **tamaño del SW** (que tan grande es el trabajo – para esto se usa la “técnica de contar”, se cuentan funcionalidades, Req, US, casos de usos)
(entra en juego la información junto con la incertidumbre que tenemos para hacer la estimacion)
(las métricas/ dtos históricos sirven para comparar el tamaño posible de nuestro SW con alguno desarrollado anteriormente con características similares)
Luego estimamos el **Esfuerzo** cantidad de horas lineales que voy a necesitar para construir el software, no incluyo qué personas lo hacen, ni calendario
Sigue el **Calendario** que es justamente calendarizar el esfuerzo y proponer una fecha de entrega.
y por ultimo se estiman los **Costos** y **Presupuestos**

Métodos utilizados para estimar

Se recomienda utilizar diferentes métodos de estimación y luego contrastar (comparar). Todos tienen un “Factor de Ajuste” que permite que el método cierre (calibraciones).
Métodos Basados En La Experiencia:
Esta técnica se basa en la experiencia pasada del equipo en proyectos similares para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.
Dentro de los basados en la experiencia:
● Datos históricos: consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sea de utilidad para futuras estimaciones, se busca que el conocimiento de cada individuo se transfiera a la organización y no estemos atados a una única persona. Se deben utilizar para no tener problemas entre desarrolladores y clientes. Esto no va acorde a la gestión ágil, que dice que la experiencia no se puede extrapolar a otros equipos.
industry data: datos de otras orgs **historical data:** datos de la org **Project data:** datos del proyecto

● Juicio experto: de los más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Existen dos técnicas para aplicarlo:
Juicio experto puro: un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (problema: si el experto se va de la organización, esta pierde su capacidad de estimación). Se puede utilizar el "método de optimista, pesimista, habitual", en el que se le pregunta al experto sobre 3 valores distintos y se aplica una fórmula para calcular la estimación.
Analogía Juicio Wideband Delphi: es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración. Hay un coordinador que va a reunir las estimaciones individuales de cada uno de los expertos cuando les dieron las especificaciones, se opina sobre las estimaciones ajenas de manera anónima, se vuelve a discutir, revisan, las vuelven a enviar al coordinador y se repite hasta que todos estén de acuerdo de forma razonable. Este método es tomado como base para el Poker Planning.

Analogía:
Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares.
Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Es decir, consiste en tomar a un proyecto e ir comparando factores, por ejemplo:
- Tamaño: es menor o mayor al proyecto anterior.
- Complejidad: más o menos complejo de lo usual.
- Usuarios: si hay más usuarios habrá más complicaciones, como la concurrencia o los datos a almacenar.

Basados exclusivamente en los recursos:
Esta técnica se basa en la cantidad y el tipo de recursos (personas, tiempo, herramientas, etc.) necesarios para llevar a cabo un proyecto. La estimación se realiza en función de la disponibilidad y costo de los recursos necesarios.

Basados exclusivamente en el mercado o en la capacidad del cliente:
Ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago. En este método no se evalúa al producto que se está intentando vender, sino a quien se lo vende. A veces se cobra menos para que nos elijan antes que a la competencia.
Esta técnica se basa en el costo de proyectos similares en el mercado para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto. Esta técnica se utiliza comúnmente en proyectos de desarrollo de software para clientes externos
(NI IDEA QUE DEF ESTARA BIEN)

Basados en los componentes del producto o en el proceso de desarrollo:
Esta técnica se basa en la identificación y descomposición de los componentes del software y del proceso de desarrollo para estimar el esfuerzo y tiempo requerido para cada componente. Luego, se suman las estimaciones para obtener una estimación total del proyecto.
Se pueden aplicar 2 enfoques:
a. Bottom-up: se descompone el producto en unidades lo más pequeñas posible. Se estima cada unidad y luego se hace la estimación total.
b. Top-down: se descompone el producto en grandes bloques y luego se estima cada uno de los componentes.

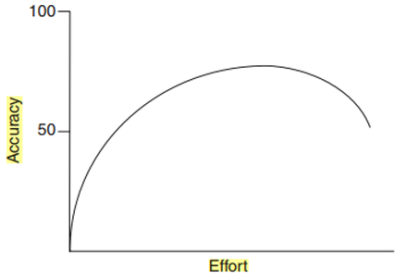
Métodos algorítmicos:
Esta técnica se basa en modelos matemáticos y estadísticos para determinar la estimación de esfuerzo y tiempo requerido para un proyecto. Estos modelos pueden tener en cuenta diferentes variables como el tamaño del software, la complejidad, la productividad del equipo, etc002E
Se ejecutan algoritmos que tienen como parámetros de entrada medidas cuantitativas de tamaño, complejidad y conocimiento de dominio. Cada variable tendrá asociada un factor multiplicativo y en función de esto se obtiene como resultado un valor de esfuerzo. Es importante destacar que, a pesar de ser un cálculo matemático, puede no ser certero igual.

Estimaciones en ambientes ágiles

Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra.
La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, por lo que el equipo de desarrollo realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones).
Se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible
Para estimar de esta forma tiene que haber libertad de opinión sin sesgar a otros y actitud de escucha
La estimacion no es algo facil, es lo más difícil luego de la Elicitacion
❗ El **beneficio** de estimar es que enriquece el trabajo del equipo, mejorando los procesos de estimacion futura y ayuda a determinar la factibilidad del trabajo planificando en etapas tempranas

Frente a la posibilidad de estimar mal y generar enojo por parte del cliente, tenemos dos reacciones posibles:
- Subestimar simplificando todo lo que hay que hacer por temor de que el costo sea alto y el cliente rechace
- Sobreestimar apropósito, si se que me lleva 2 hs digo 4 por si acaso ocurre un imprevisto

● El enfoque agil sostiene que no hay q enfocar muchísimo esfuerzo en estimar para poder acercarnos a la realidad, ningún extremo es bueno



Como se puede ver en la imagen el punto ideal sería al medio
Un punto medio en el esfuerzo daría la mejor precisión
(de todas formas lo mejor sería estar lo más posible del lado izq. representando poco esfuerzo y que de la mayor precisión posible)

¿Las estimaciones agiles o estimaciones en general?:
- Son medidas relativas y no absolutas
- No es una medida basada en tiempo
- Se emplea mejor el tiempo de análisis de las Stories.
- mejor comparar no absoluto

Estimaciones relativas y absoluta

Las estimaciones relativas consiste en estimar mediante comparación porque el humano es malo para **comparar de forma absoluta** (dar directamente números) por lo tanto **estima relativamente** comparando entre cosas

Para estimar se deben tener en cuenta tres elementos: complejidad, esfuerzo e incertidumbre.
Estas tres variables son las que conforman al Story Point y de esta manera podemos comparar las US

¿Qué se estima?

Lo que **se estima** en ambientes ágiles, es solo **el tamaño de las US** (o Features) usando como unidad de medida los **Story Points**, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo.
Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas ej: serie de Fibonacci.
Al ser estimaciones de tamaños relativos, (se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino que las estimaciones sean consistentes en el conjunto ==> para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, lo que importa es que una US de 5 puntos, siempre sea más chica que una de 8.
Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de **Timebox**). Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.
el líder de proyecto es quien estima en el enfoque tradicional

Anotación Fnk = La estimacion del esfuerzo se utiliza para derivar a la estimacion de tiempo y poder responder cuando va a estar y al final se realiza una estimación de costo
esto es para un enfoque agil

Tamaño vs. Esfuerzo
Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario.
La **complejidad es independiente de quien haga el proyecto**, y el **esfuerzo depende específicamente de la persona que lo va a llevar a cabo**, conlleva horas lineales (horas únicamente de trabajo neto, una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales).
Por otro lado, no se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocidad
Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.
Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración es o completa o no.
Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.
Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones.
Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo. Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Reestimación
Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que en el proceso de estimación se han detectado otras U.S. con tamaños más grandes o pequeños. (tiene que ver con mantener la consistencia de tamaños).
No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

Planificación
En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios, para encauzar los esfuerzos de manera correcta, y poder lograr los objetivos planteados.
“Planning is everything. Plans are nothing” → esto apunta a que lo importante de planificar, es el proceso en sí. No el hecho de realizar una documentación exhaustiva que respete determinados templates. Sí debe quedar registrado de alguna manera, idealmente bajo templates, pero sólo para servir de ayuda de memoria. Lo importante es el proceso.

Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles:

Planificación de Release

Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.
Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto

Planificación de Iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada.
Esto se hace teniendo en cuenta el tiempo (que es una variable constante por ser metodología ágil) y la velocidad de trabajo del equipo (el cual se mantiene constante también). Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Tamaño

Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para completar una US/feature.
El tamaño no es lo mismo que el esfuerzo, ya que este último indica las horas humano lineal requeridas para el desarrollo de la US ==> entonces una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando, debido a que el esfuerzo varía dependiendo de los individuos.
Formas para estimar el tamaño:
- del 1 al 10
- Talles de remeras
- Exponencial de base 2: 2,4,8,16,32,64, …
- Fibonacci: 0,1,1,2,3,5,8,13, …

Una vez definida la escala, no se cambia. Y si de cambia, también se debe cambiar el patrón de medición.

Story Point (de nuevo)

Unidad de estimación que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una US en términos relativos respecto a otra US.
Da la idea del "peso" de cada US, decide cuán grande/compleja es. Por lo general esa complejidad tiende a incrementarse exponencialmente.
- **Complejidad**: que tan complicada des de implementar la US
- **Esfuerzo**: debe estimarlo quien va a llevar a cabo el trabajo, es dependiente de c/persona, son Hs lineales
- **Incertidumbre**: es cuánta información falta para ser más adecuados en la estimación
    - Técnica: cuando se tiene dudas sobre el dominio de la solución.
    - De negocio: incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

¿Cómo se estima la duración del proyecto?

Si la estimación se realiza utilizando Story Points para las User Stories, para determinar la duración de un proyecto se realiza la derivación tomando el número total de story points de sus US y dividiéndolas por la velocidad del equipo.

Poker Planning o Poker Estimation

Técnica de estimación en ambientes ágiles.
Combinación de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2.
En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen. En otras palabras, el equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Se decidió usar la escala Fibonacci para los Story points
- **Valor 1 o 1/2**: son los valores más bajos y se aplican a una funcionalidad pequeña.
- **Valor 2- 3**: funcionalidad pequeña a mediana.
- **Valor 5**: funcionalidad media.
- **Valor 8**: funcionalidad grande, donde nos preguntamos si es posible dividir.
- **Valor 13 o más**: necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento

1. Se define una lista de actividades, módulos, casos de uso, Users stories, etc.

2. Se acuerda cuál de las anteriores es la que tiene menor grado de incertidumbre y se la define como canónica asignándole 1 o 2 story point. Esta User es utilizada como referencia para comparar.
Se puede elegir canónicas de 3 SP, para tener un margen para hacia arriba y hacia abajo.
También él ½ se incluye en la serie.

3. Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.

4. Utilizando juicio experto, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones. Luego el resto del medio aportan sus opiniones. Siempre la escucha es activa, para poder aprender.

5. En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución del paso 4.

6. Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Consideraciones

1. Para realizar una estimación se debe tener en cuenta el criterio de Done y no solo la tarea de programación, es decir todo lo que hay que hacer para terminar una US
2. El tiempo para realizar estimaciones, como cualquier otra actividad en los ambientes ágiles, es time-box.
3. Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Estimaciones en proyectos pequeños

Para estimar proyectos pequeños lo mejor es utilizar datos históricos propios de la organización, dado que en estos casos los datos históricos de la industria no se adaptan correctamente a estas situaciones. Las estimaciones en proyectos pequeños dependen en gran medida de las capacidades de los individuos que hacen el trabajo. El SEI propone el Personal Software Process para proyectos unipersonales o 2 personas, conocidos como "mini proyectos".

Diferencias con las estimaciones en ambientes tradicionales

1. Respecto a las tres dimensiones
En Agile se tiene fijo el tiempo (lo que dura la iteración) y los recursos afectados para trabajar, dejando variable el alcance. al estimar en estos ambientes se responde a cuánto características de producto se puede comprometer el equipo a construir en la iteración. Se entiende, además, que en estos ambientes el producto no está completamente definido, sino que se tiene una visión de este.
En los ambientes tradicionales, el alcance es lo que está fijo = producto definido, tiempo y recurso variable.
Por esta razón en los procesos definidos se estima primero el tamaño, ya que es la base para estimar el resto.

2. Respecto a la precisión
Las estimaciones en los ambientes tradicionales buscan ser más precisas, debido a que son las bases para luego planificar.
Dado que la precisión es realmente cara, la precisión en los ambientes ágiles es considerada un desperdicio.

3. Principal diferencia
Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas.

4. Quien estima
En la estimación tradicional, las estimaciones las realiza el líder del proyecto o en ocasiones las realiza un experto, por lo que su estimación no es representativa de la cantidad de trabajo que requiere una persona del equipo para terminar una funcionalidad del producto. Esta es una de las razones principales de los desvíos en los proyectos de software tradicional.
En los ambientes ágiles, es el equipo el que realiza la estimación. Lo óptimo es que cada uno estime su propio trabajo y de esta manera, es posible disminuir la probabilidad de desviaciones.

5. Confusión con la planificación
En los ambientes tradicionales existe un gran error que es asumir las estimaciones como compromisos, como si fueran planificaciones. Esto implica que las estimaciones se transforman en promesas. Mientras que en agile, asumen que las estimaciones están inmersas en el campo de las probabilidades y por lo tanto no implican un compromiso. Una falla en la estimación no implica extender el tiempo, ya que en ágiles trabajamos con el tiempo fijo, por lo tanto, lo que se ajusta es el alcance.

6. Momentos para estimar
En el enfoque tradicional se estima al comienzo del proyecto, luego de la especificación de requerimientos y luego del diseño. Realmente se estima cada vez que se modifica el alcance, ya que es la variable que se mantiene fija y de la cual se deriva el resto.
En ágiles se estima al comienzo de cada sprint.

SCRUM

Scrum es un marco de trabajo ligero que ayuda a personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejo.
Consiste en eventos iterativos que permiten agregar valor a través de generar un producto
¿se enfoca en la? Creación de productos complejos de manera **adaptativa** ya que no se sabe con precisión que producto vamos a crear.
❗ Scrum da pautas sobre cómo organizar el trabajo, NO sobre cómo desarrollar software
❗ NO es una metodología NI un proceso completo, sino que se usan ciertas practicas de scrum como base para desarrollar un proceso completo

Scrum requiere un Scrum Master para fomentar un entorno donde:
- Un PO ordena el trabajo del product backlog
- El equipo Scrum convierte una selección de trabajo en un incremento de valor durante un sprint
- El equipo de scrum y stakeholders inspeccionan resultados y realizan ajustes necesarios p/el prox sprint
- Repetir durante las iteraciones

Scrum se basa en el empirismo, el cual afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa
Scrum emplea un **enfoque iterativo e incremental** para optimizar la previsibilidad y controlar el riesgo.
Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir habilidades según sea necesario.
Scrum combina cuatro eventos formales para la **inspección y adaptación** dentro de un evento contenedor, el Sprint. Estos eventos funcionan porque implementan los pilares empíricos de Scrum: **transparencia, inspección y adaptación**.

Pilares de Scrum

Transparencia: El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo.
Con Scrum, las decisiones importantes se basan en el estado percibido de sus tres artefactos formales. Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo. La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

Inspección: Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables. Para ayudar con la inspección, Scrum proporciona cadencia en forma de sus cinco eventos. La inspección permite la adaptación. La inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios.

Adaptación: Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional.
La adaptación se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Valores de Scrum

El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores:
Compromiso, Enfoque, Apertura, Respeto y Coraje

El equipo de Scrum se **compromete** a lograr sus objetivos y apoyarse mutuamente. Su **enfoque** principal es el trabajo del Sprint para hacer el mejor progreso posible hacia objetivos. El equipo y sus partes interesadas están **abiertos** sobre el trabajo y los desafíos. Los miembros del equipo de Scrum se **respetan** mutuamente, Los miembros del equipo de Scrum tienen el valor de hacer lo correcto y de trabajar en problemas complejos. **(Coraje)**
Estos valores dan dirección al equipo de Scrum con respecto a su trabajo, acciones, comportamiento, decisiones que se toman, y la forma en que se usa Scrum deben reforzar estos valores no disminuirlos
Cuando estos valores son asimilados por el equipo de Scrum y las personas con las que trabajan, los pilares empíricos de Scrum de transparencia, inspección y adaptación cobran vida construyendo confianza.

Equipo Scrum

Son pequeños equipos de personas, tienen un scrum master, un PO y desarrolladores, no hay subequipos ni jerarquías, es una unidad cohesionada de profesionales enfocados en un obj a la vez el obj del producto
Son multifuncionales básicamente cada miembro tiene las habilidades necesarias p/crear valor en cada sprint
Son autogestionados = internamente deciden quien hace que cuando y como

Son lo suficientemente pequeños como para permanecer agil y lo suficientemente grandes como para completar un trabajo significativo dentro de un sprint (10 o menos)
Los equipos más pequeños se comunican mejor y son más productivos. Si se vuelven demasiado grandes, se debe considerar la posibilidad de reorganizarse en varios equipos Scrum cohesionado
El equipo Scrum es responsable de todas las actividades relacionadas con el producto, desde la colaboración, verificación, mantenimiento, operación, experimentación, investigación y desarrollo, y cualquier otra cosa que pueda ser necesaria
Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint. Scrum define tres responsabilidades específicas dentro del equipo de Scrum: desarrolladores, PO y el Scrum Master.

Desarrolladores

Personas del equipo Scrum que se comprometen a crear cualquier aspecto de un Incremento útil (funcional) en cada Sprint.
Las habilidades específicas que necesitas son amplias y varían con el dominio del trabajo siempre son:
- Crear un plan para el sprint (sprint backlog)
- Inculcar la **calidad** adhiriéndose a una DoD
- Adaptar su plan cada día hacia el Objetivo Sprint
- Responsabilidad profesional

Product Owner

Es el responsable de maximizar el valor del producto resultante del trabajo del equipo. La forma en que esto se hace esto puede variar entre organizaciones, equipos Scrum e individuos.
También es responsable de la gestión eficaz del Product Backlog, incluye:
- Desarrollar y comunicar explícitamente el Objetivo del Producto;
- Creación y comunicación clara de elementos de trabajo pendiente del producto;
- Pedido de artículos de trabajo pendiente del producto;
- Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.
El PO puede hacer el trabajo o delegarlo, pero va a seguir siendo responsable
Para tener un PO exitoso toda la Org debe respetar sus decisiones
El PO es una persona no un comité, representa las necesidades de muchas partes interesadas

Scrum Master

Responsable de establecer scrum tal como lo define la guía de scrum, ayuda a todos a comprender la teoría y practica de scrum tanto dentro del equipo como en toda la organización
Son lideres que sirven al equipo y a toda la org, alguna de sus responsabilidades son:
- Capacitar a los miembros del equipo en autogestión y multifuncionalidad;
- Ayudar al equipo Scrum a centrarse en la creación de incrementos de alto valor que cumplan con la DoD;
- Promover la eliminación de los impedimentos para el progreso del equipo Scrum;
- Asegurar que todos los eventos de Scrum se lleven a cabo, sean positivos, productivos y que se respete el tiempo establecido (time-box) para cada uno de ellos.

El scrum master sirve al PO en:
- Ayudar a encontrar técnicas para una definición eficaz de objetivos del producto y gestión de retrasos en el producto;
- Ayudar al equipo de Scrum a comprender la necesidad de elementos de trabajo pendiente de productos claros y concisos;
- Ayudar a establecer la planificación empírica de productos para un entorno complejo;
- Facilitar la colaboración de las partes interesadas según sea solicitado o necesario

El scrum master sirve a la organización:
- Liderar, capacitar y mentorizar a la organización en su adopción de Scrum;
- Planificar y asesorar sobre la implementación de Scrum dentro de la organización;
- Ayudar a las personas y a las partes interesadas a comprender y promulgar un enfoque empírico
- Eliminar las barreras entre las partes interesadas y los equipos de Scrum

Evento Scrum

El Sprint es un contenedor para todos los eventos. Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum, Estos eventos están diseñados para permitir la transparencia necesaria.
Si no se realizan los eventos según lo prescrito, se pierden oportunidades para inspeccionar y adaptarse.

Sprint

Latido de corazón de scrum, donde las ideas se convierten en valor
Son eventos de longitud fija de un mes o menos para crear consistencia.
Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior. Todo el trabajo necesario para alcanzar el objetivo del producto:
- La Planificación (Sprint Planning),
- Daily Scrums,
- Revisión del Sprint (Sprint Review)
- La Retrospectiva (Sprint Retrospective),

ocurren dentro del Sprints.
Durante el sprint:
- No se hacen cambios que pongan en peligro el objetivo de sprint
- La calidad no disminuye
- El trabajo pendiente del producto se refina según sea necesario

Los Sprints permiten la previsibilidad al garantizar la inspección y adaptación.
Cuando el horizonte de un Sprint es demasiado largo, el Objetivo de Sprint puede volverse obsoleto, la complejidad puede aumentar y el riesgo puede aumentar. Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un período de tiempo más pequeño. Cada Sprint puede considerarse un proyecto corto.
Existen varias practicas para pronosticar el progreso, gráficos burn-down, burn-up, flujos acumulativos, estos no sustituyen la importancia del empirismo
Un Sprint podría ser cancelado si el Objetivo del Sprint se vuelve obsoleto. Solo el Propietario del Producto tiene la autoridad para cancelar el Sprint.

Planificación de sprint
El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará.
Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum. El PO se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importantes y cómo se asignan al objetivo del producto
El objetivo de sprint (sprint goal), elementos de trabajo pendiente, plan ¿de sprint? Se conoce como trabajo pendiente de sprint (sprint backlog)
en la planificación del sprint se aborda:
- **¿Por qué este sprint es valioso?** El PO propone como el producto podría aumentar su valor y utilidad en el sprint actual, todo el equipo colabora para definir un objetivo de sprint que comunique porque el sprint es valioso para las partes interesadas, el obj debe finalizarse antes del final de la planificación del sprint

- **¿Qué se puede hacer este sprint?** Se debate con el PO, los desarrolladores eligen elementos del product backlog p/incluir en el sprint actual, el equipo refina esos elementos aumentando la comprensión y confianza

- **¿Cómo se realizará el trabajo elegido?**
Para cada elemento de trabajo pendiente de producto (Product Backlog item) seleccionado, los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con la DoD

El Sprint Planning tiene una duración máxima de ocho horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

Scrum diario
El daily scrum consiste en **inspeccionar** el progreso hacia el obj de sprint y **adaptar** el sprint backlog según sea necesario, es un **evento de 15min máximo** donde el equipo busca reducir complejidad, se lleva a cabo al mismo tiempo y lugar todos los días laborales del sprint.
Si el PO o scrum master están trabajando activamente pueden participar como desarrolladores
El daily scrum mejora la comunicación, identifica impedimentos, promueve la rápida toma de decisiones eliminando la necesidad de otras reuniones

Revisión del Sprint
Consiste en inspeccionar el resultado del sprint y determinar futuras adaptaciones. El equipo presenta los resultados de su trabajo a los stakeholders clave y se discute el progreso hacia el Obj de Producto.
Durante el evento, el equipo y los stakeholders revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. En base a esta información, los asistentes colaboran en qué hacer a continuación. El trabajo pendiente del producto también se puede ajustar para satisfacer nuevas oportunidades
La sprint review debe limitarse a ser una simple presentación
La revisión de sprint tiene una duración máxima de cuatro horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto

Retrospectiva del Sprint
Su propósito es planificar formas de aumentar la calidad y la eficacia.
El equipo inspecciona como fue el ultimo sprint con respecto a individuos, interacciones, procesos, herramientas y DoD
Se identifica y exploran los orígenes de las suposiciones que desviaron el dom de trabajo
El equipo analiza que fue bien en el sprint, que problemas hubo y como fueron o no resueltos
Se identifican los cambios más útiles para mejorar la eficacia
Duración máxima de tres horas

Prioridad ≠ importancia
Importancia: indica qué tanto valor o impacto tiene algo.
Prioridad: indica qué tan pronto o antes que otras cosas debe hacerse.

Artefactos Scrum

Representan el trabajo o valor.
Están diseñados para maximizar la transparencia de la información, por lo que cada persona que tiene que inspeccionar parten de la misma base/información para adaptar
Cada artefacto contiene un compromiso que le da un propósito, permite saber si estamos avanzando correctamente y garantiza que proporciona información que mejora la transparencia
- Para el Product backlog es el objetivo del producto.
- Para el Sprint Backlog es el Sprint Goal.
- Para el Incremento es la Definición de Hecho.

Product backlog

Lista emergente y ordenada de lo que se necesita realizar para mejorar el producto, única fuente de trabajo emprendida por el equipo
Los elementos de trabajo pendiente de producto que puede ser hecho por el equipo dentro de un Sprint se consideran listos para su selección en un evento de planificación de Sprint. Por lo general adquieren este grado de transparencia después de las actividades de refinación. El refinamiento de Backlog del producto es el acto de descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos. Esta es una actividad en curso para agregar detalles, como una descripción, un pedido y un tamaño.

Compromiso: Objetivo del producto (Product Goal)

El objetivo del producto describe un estado futuro del producto que puede servir como objetivo para el equipo contra el cual planificar. El objetivo del producto se encuentra en el Product Backlog.
El resto del trabajo pendiente del producto surge para definir "qué" cumplirá el objetivo del producto.

La pila del Sprint (Sprint Backlog)

Es el plan de trabajo del equipo para ese sprint
Dentro de un sprint backlog tenemos 3 cosas:
1. Porqué = sprint goal: Objetivo que queremos conseguir durante el sprint
(sprint goal: permitir al cliente simular un crédito)

2. Qué = elementos seleccionados del product backlog: Son las cosas que tenemos que hacer para alcanzar el objetivo (crear formularios/calcular intereses/calcular cuotas/mostrar resultados)
Estas cosas vienen del Product Backlog y las seleccionamos para este Sprint.

3. Cómo = plan para realizar el trabajo: Es cómo los desarrolladores van a llevar a cabo esas tareas.
(hacemos el formulario → implementamos el cálculo → hacemos las pruebas → integramos todo)

Compromiso: Sprint Goal

El Sprint Goal es el único objetivo para el Sprint. Aunque el objetivo de Sprint es un compromiso de los desarrolladores, proporciona flexibilidad en términos del trabajo exacto necesario para lograrlo. El Objetivo Sprint también crea coherencia y enfoque, animando al equipo de Scrum a trabajar juntos en lugar de en iniciativas separadas.
El objetivo de Sprint se crea durante el Sprint Planning y, a continuación, se agrega al Trabajo pendiente de Sprint. A medida que los desarrolladores trabajan durante el Sprint, tienen en cuenta el objetivo de Sprint. Si el trabajo resulta ser diferente de lo que esperaban, colaboran con el propietario del producto para negociar el alcance del Trabajo pendiente de Sprint dentro del Sprint sin afectar al objetivo de Sprint.

Incremento (Increment)

Un Incremento es un paso hacia el Objetivo del Producto.
Cada Incremento es aditivo a todos los Incrementos anteriores y verificado a fondo, asegurando que todos los Incrementos funcionen juntos.
Para proporcionar el valor, el incremento debe ser utilizable.
Se pueden crear varios incrementos dentro de un Sprint. La suma de los Incrementos se presenta en la Revisión Sprint.
Sin embargo, un Incremento puede ser entregado a las partes interesadas antes del final del Sprint. La revisión de Sprint nunca debe considerarse una puerta para liberar valor. El trabajo no se puede considerar parte de un Incremento a menos que cumpla con la Definición de Hecho.

Compromiso: Definición de Hecho (Definition of Done)

La Definición de Hecho es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto. En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento.
La definición de Hecho crea transparencia al proporcionar a todos una comprensión compartida de qué trabajo se completó como parte del Incremento. Si un elemento de trabajo pendiente de producto no cumple con la definición de hecho, no se puede liberar, ni siquiera presentar en la revisión de Sprint. En su lugar, vuelve al Trabajo pendiente del producto para su consideración futura.
Los desarrolladores deben ajustarse a la definición de Hecho. Si hay varios equipos de Scrum trabajando juntos en un producto, deben definir y cumplir mutuamente con la misma definición de hecho.

TimeBox



Existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos
#Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos.
De esta manera, nos volvemos más confiables, con un ritmo de trabajo sostenible. Todas las ceremonias son Timebox. El refinamiento del Product Backlog al ser una actividad continua, no tiene definido un tiempo exacto, sino que el tiempo se expresa en términos porcentuales sobre la definición del Sprint.

- El refinamiento del product backlog NO pertenece al ciclo scrum
- El modelo INVEST
- La calidad del output es INEGOCIABLE

Capacidad del equipo en un sprint

La capacidad es una métrica q utiliza Scrum P/determinar la cantidad de trabajo a la que se puede comprometer un equipo en un sprint
Es estimable y se mide
Una de las cosas que se hace en el Sprint Planning es determinar la capacidad del equipo, luego se contrasta cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.
Para equipos más maduros, la capacidad puede ser estimada en Story Points y para equipos menos maduros se puede estimar en horas ideales.

Niveles de Planificación

- **Portfolio**: 1año o más - **Producto**: +9 meses - **Release**: ± 3 – 9 meses
- **Iteración**: 1 semana – 1 mes - **Día**: diariamente

Herramientas de Scrum

Tarjeta de tarea

Tarjeta con 5 partes:
Codigo de item backlog (identificador del item
nombre de la actividad / frase verbal de lo que hay q hacer
Iniciales del responsable de realizar la tarea.
Número de día del sprint.
Estimación del tiempo para finalizar la tarea.

Tablero

Tablero utilizado por el equipo de desarrollo donde se colocan las tarjetas a realizar durante el sprint
se divide en 5 secciones:
1. Story: el nombre de la historia.
2. To Do: tareas por hacer en el Sprint.
3. Work In Process: tareas que se están realizando.
4. To Verify: tareas finalizadas que deben verificarse.
5. Done: tareas terminadas: finalizadas y verificadas.

Gráficos del Backlog

Gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto.

Sprint Burn-Down Chart

Visualiza con una curva descendente cuánto trabajo queda para terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el sprint.

Sprint Burn-Up Chart

Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.

Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.





Frameworks para escalar SCRUM

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de escalar este Framework para poder gestionar productos de mayor envergadura, ya que las practicas agiles se utilizan en ambientes complejos para reducir tal complejidad.
nexxus - less

Gestión de productos de software

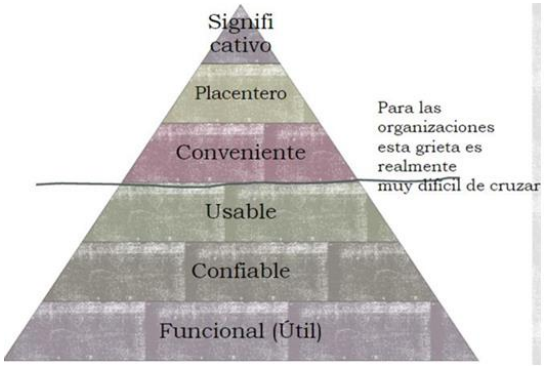
Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos.
Cuando creamos un producto debemos enfocarnos en aquello que es de valor para el cliente, sabiendo que el 7% de las características del software son siempre usadas (la mayor parte del valor agregado está concentrado ahí).
Alrededor del 80% de las características del software que se desarrolla no se utilizan. El desafío está en encontrar ese 7%.

La org q usa el software se enfoca en lo superficial, aquello que considera significativo y placentero, pero no se fija si el producto es funcional o confiable, lo cual debe ser la base del producto para proveer una experiencia que cumpla con todas las expectativas de un cliente

Creamos productos para, satisfacer a clientes, obtener dinero, cambiar al mundo y tener muchos usuarios logueados que funciona como métrica para saber que tan aceptado es un producto

Evolución de los productos de software

Cuando vamos a desarrollar un software la mayoría de las veces se empieza a construir un producto sobre la base de lo funcional, que haga lo que tiene que hacer.
Luego si se quiere avanzar más, el software debe ser confiable donde los usuarios no corran peligro usando el producto y los resultados que este producto me da yo pueda depender de ellos sin ningún problema.
Luego, la usabilidad tiene que ver con aspectos del UX/UI, donde el usuario podrá lograr sus objetivos productivamente usando este producto, estando a gusto con su uso.
Tiene que ser productivo, no es lo mismo que la utilidad. Por ejemplo, si tarda 12 horas en realizar una operación donde el anterior tarda 4 horas no es productivo para mí y no mejora la calidad de vida al usuario.



Existe una línea en la pirámide que separa la usabilidad de la conveniencia. La mayoría de los productos no se ubican por encima de esta línea, debido a que resulta complejo que un software sea conveniente para el desempeño de tareas de ciertas personas, y que estas no sean “esclavos” del software, limitándose a cumplir con las características de funcional, confiable y/o usable. Placentero y significativo para todo el mundo, algo que nos cambie la forma de usar algo o de vivir.

Para que un SW sea conveniente hay que estar en un nivel superior a que sea simplemente usable.
Un software usable es uno que permite al usuario hacer lo que necesita de una manera razonablemente cómoda. Pero eso no significa necesariamente que el software haya cambiado para mejor la forma en que trabajo.
Un software es conveniente cuando no solamente permite h acer una tarea, sino que genera una mejora significativa en la vida o en el trabajo del usuario.
un software puede ser: "Funcional, confiable y usable, pero el usuario tiene que adaptarse al software."
El sistema funciona y quizás es usable, pero el usuario está trabajando alrededor de las limitaciones del software.

Planificación del producto ágil

En las metodologías ágiles, se tiene una visión del producto materializado en el Product backlog, que contiene todas las características de la funcionalidad que requiere el producto que han sido identificadas hasta el momento.
Esta visión se descompone en releases (entregas frecuentes al cliente). Cada release se construye a partir de sucesivas iteraciones.

Valor vs desperdicio

La productividad de un Startup no puede medirse en términos de cuánto se construye cada día, por el contrario, se debe medir en términos de construir lo correcto cada día y lo correcto es aquello que agrega valor. Si no construimos valor, entonces estamos invirtiendo esfuerzo en desperdicio.

Mínimo Producto Viable

Es la versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo.
Detrás de este concepto esta la idea de que ver lo que la gente realmente hace con el producto es mucho más confiable que preguntarle a la gente qué harían.
Una premisa clave detrás de la idea del MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio.

Entre sus características se destaca que:

1. Tiene el valor suficiente para que las personas estén dispuestas a usarlo/comprarlo.
2. Demuestra beneficios a futuro para retener a los usuarios.
3. Proporciona un ciclo de retroalimentación para guiar el desarrollo a futuro.

Errores comunes son:

1. Confundirlo con MMF o MMP

MMF (Característica Comerciable Mínima) Funcionalidad mínima que ya tiene valor para el cliente y puede comercializarse/entregarse.
MMP (Producto Comerciable Mínimo) Producto mínimo que ya tiene suficiente valor para salir al mercado.
MVP (Mínimo Producto Viable) Producto mínimo para validar una idea y aprender de los usuarios.
a diferencia del MVP, MMF y MMP se enfocan en ganar, en cambio un MVP se enfoca en el aprendizaje

2. Enfatizar la parte de mínima y dejar de lado la parte de viable. Si el producto no tiene la calidad suficiente, no será posible obtener la retroalimentación del cliente.
3. Hacer del MVP un producto final, es decir, entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.

Un MVP permite validar diferentes hipótesis:

1. Hipótesis del valor: prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es correcto).

Una métrica de prueba: tasa de retención (más de la mitad de los usuarios volvieron a utilizar el sistema todos los días, solo un 25% dejó de usarlo).

2. Hipótesis de crecimiento: prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto).

Una métrica de prueba: tasa de referencia o Net Promoter Score (NPS) (Se corrió la voz acerca de lo que era, no fue siquiera necesario el uso de políticas de marketing o publicidad).

Preparar un MVP



1) Encontrar un nicho de mercado.
2) **Roadmap**: definición de muy alto nivel de qué características del producto se podrían tener a cierta altura del año. Esto sirve para los inversores.
Alto nivel = forma general
Bajo nivel = detallado y especifico
3) **Investigar la competencia**: ver hasta dónde llega, qué hace, qué fortalezas tiene.
4) Pre-vender el MVP.
5) Testear las distintas suposiciones.
6) Asegurarse de que el MVP esté resolviendo el problema correcto.
7) Enfocarse solo en las funcionalidades principales.

Design Thinking

Es un enfoque para la creación de servicios digitales centrada en el cumplimiento de las expectativas del usuario mediante un proceso de mejora continua que advierte las verdaderas necesidades de este.
Para llevar a cabo su proceso, utiliza un esquema de pensamiento divergente y convergente.
El primero busca tener una visión amplia del espectro de soluciones que responden al problema inicial.
El segundo consiste en reducir a la mejor idea el conjunto de posibles soluciones, con el objetivo de brindar aquella que es más adecuada para las necesidades del cliente.

El proceso cuenta con 5 pasos y es iterativo (no lineal y se puede volver a los pasos anteriores las veces que sea necesario):
1. Empatizar: entender las necesidades del cliente mediante la utilización del razonamiento intuitivo.
2. Definir: determinar lo valioso para el cliente a partir de lo recopilado durante la primera etapa, lo cual permitirá guiar el trabajo.
3. Idear: articular muchas soluciones posibles. Cada miembro del equipo aporta una mirada diferente para generar muchas ideas y luego filtrar las más viables.
4. Prototipar: consiste en materializar las ideas. Las ideas seleccionadas se transforman en prototipos que permitan visualizar la solución propuesta.
5. Probar: es una de las etapas más importantes del proceso, ya que permite validar los prototipos con los usuarios implicados en la solución, obteniendo retroalimentación y permitiendo iterar.

Lean UX

La filosofía Lean UX es un proceso iterativo, basado en: construir, medir, aprender.
Se trata de sacar conclusiones basándonos en los usuarios reales que usan (van a usar) el producto/servicio.
Si desde el primer momento el usuario no «aprueba» el producto, es más barato pivotar y volver a iniciar el proceso. Es decir, la metodología Lean UX nos permite «equivocarnos» en estadios muy precoces, cuando aún es muy barato reconducir la idea.

Explicación MVP, MVF, MMF

1. Comprender que un producto nuevo tiene una hipótesis de valor único: el Producto será único (UVP).

2. El siguiente paso es crear un producto mínimo viable (MVP) para probar su hipótesis.
Esto se centra en su propuesta de valor única, normalmente proporciona un par de funciones para asegurarse de que sea "viable" como producto.

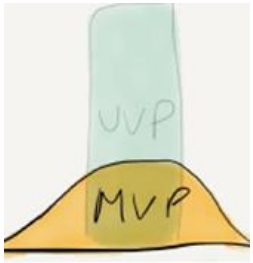
3. Tu MVP también es una hipótesis.
Podría ser lo suficientemente bueno para encontrar un mercado o no. Se muestra el caso en el que cada cliente potencial con el que interactúas te dice "Esto es genial, pero para poder usarlo necesito X" y X es diferente para cada cliente / usuario. Esto muestra que aún no se encuentra un mercado para el producto.

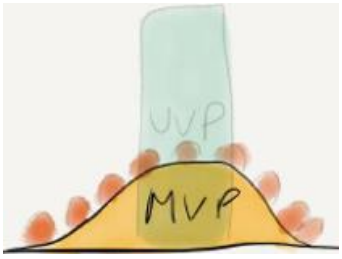
4. Si por el contrario ves cada vez más respuestas apuntando al MISMO X entonces tiene sentido revisar la hipótesis de Cliente/Problema/Solución.
Básicamente, ejecutar un pivot, construyendo MVP2 centrado en la nueva hipótesis basada en el aprendizaje reciente generado por el anterior MVP y la retroalimentacion.

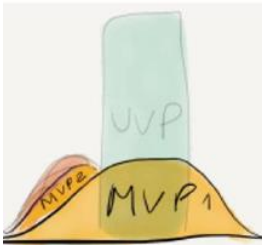
5. Supongamos que MVP2 es exitoso y está viendo una atracción real de los primeros usuarios.
Si se desea aumentar el crecimiento y buscar una penetración más profunda de sus primeros usuarios además de atraer nuevos clientes.
En función del feedback recopilado y la investigación de su gestión de productos, tiene un par de áreas que potencialmente pueden traer este crecimiento. Algunos de ellos por cierto amplían su propuesta de valor única y algunos hacen que su producto actual sea más robusto. ¿posibles MMF?

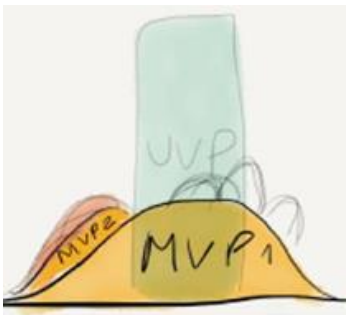
6. En el caso de áreas con una fuerte indicación de valor, puede directamente definir un MMF (Características mínimas comercializables).
Para encontrar la pieza mínima que pueda empezar a traer crecimiento. El objetivo del MMF es aportar valor. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas. Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área













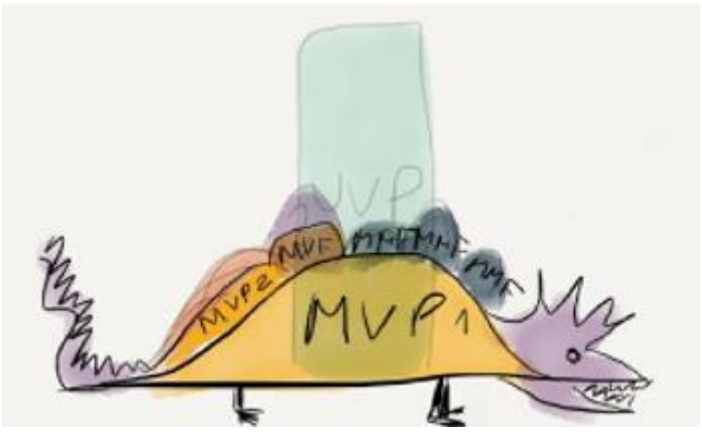
7. Si se desea esperar para ver si el primer MMF apesta ... entonces está de vuelta en la tierra de la hipótesis. Ahora tu hipótesis se centra en una característica en lugar del producto. Tienes un área con alto potencial pero también alta incertidumbre.



8. La forma de afrontarlo es crear una función "pionera": MVF (Característica Mínima Viable). La característica mínima que aún puede ser viable para uso real y aprendizaje de los usuarios reales. Si el MVF resulta exitoso (hit gold), puede desarrollar más MMF en esa área para tomar ventaja (si eso tiene sentido). Si no es así, puede cambiar a otro enfoque hacia esa área de características, o en algún momento buscar una ruta de crecimiento alternativa. Esencialmente, el MVF es una versión mini del MVP.



9. El producto se cultiva en mercados inciertos al intentar varios MVP. Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio / requisitos en las áreas en las que se está enfocando.
El dinosaurio carpaccio se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a reducir el riesgo de ejecución.
Esas porciones más pequeñas pueden tener un valor comercial tangible o no. Para otros puede ser importante proporcionar una temprana retroalimentación de decisiones a lo largo del camino.



Productos mínimos para la gestión de productos

MVP — Minimum Viable Product: producto mínimo para validar una hipótesis/aprender.
MMF — Minimum Marketable Feature: funcionalidad mínima que aporta valor comercial.
MMP — Minimum Marketable Product: producto mínimo con valor suficiente para salir al mercado.
MMR — Minimum Marketable Release: versión mínima lista para ser liberada a los usuarios.
MVE — Minimum Viable Experience: experiencia mínima necesaria para validar si la experiencia del usuario funciona.
MLP — Minimum Lovable Product: producto mínimo que, además de funcionar, busca que los usuarios lo valoren/gusten de él.
MVP+ / MVP 2.0: evolución de un MVP incorporando lo aprendido de los usuarios; no es un concepto tan estandarizado.

MVP: Es una versión de un nuevo producto creado con el menor esfuerzo posible que esta dirigido a clientes potenciales, se usa para obtener aprendizaje validado, es mas cercano a un prototipo que a una versión real

MMF: Es la pieza más pequeña de funcionalidad que puede ser liberada es una versión mini de MVP
Tiene valor para la org y los usuarios, es parte de un MMR o MMP

MMP: Primer release de un MMR dirigido a primeros usuarios, Focalizado en características clave que satisfarán a este grupo clave.

MMR: Release de un producto que tiene el conjunto de características más pequeño posible El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales

Gestión del Software como producto

Los sistemas de software cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para no perder la trazabilidad de los cambios que se incorporan a cada versión.

Software Configuration Management (Gestión de Configuración de Software)

Disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo (administrativo y técnico) a:
● **Identificar** características técnicas y funcionales de ítems de configuración;
● **Documentar** características técnicas y funcionales de ítems de configuración;
● **Controlar los cambios** de esas características identificadas y documentadas;
● **Registrar y reportar** estos cambios;
● **Verificar correspondencia** con los requerimientos (trazabilidad).



Su propósito es resolver problemas de diferente índole, a través del establecimiento y mantenimiento de la integridad del producto de software a lo largo de todo su ciclo de vida.
Algunos de esos problemas son:
- Pérdida de componentes;
- Pérdida de cambios (la versión del componente no es la última);
- Regresión de fallas;
- Doble mantenimiento;
- Superposición de cambios;
- Cambios no validados.

Básicamente consiste en establecer y mantener la integridad del SW a lo largo de su ciclo de vida implicando identificar la configuración en un momento dado, controlando sistemáticamente cambios

La calidad es subjetiva además el SW es intangible por lo que es difícil medir si la calidad responde a las expectativas

La **integridad** es el medio por el cual podemos garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.
La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.
Decimos que se mantiene la integridad de un producto de software cuando:
● Satisface las necesidades de usuario: hace lo que el usuario espera que haga;
● Permite la rastreabilidad durante todo su ciclo de vida:
existen vínculos o conexiones entre los ítems de configuración (los cuales deben ser definidos desde un primer momento del ciclo de vida del producto) permiten analizar en donde impactará un cambio en un ítem de configuración. De esta forma, se puede determinar cómo impactará cada cambio de requerimientos a través de la trazabilidad. A mayor cantidad de vínculos → mayor información → mayor trazabilidad → mayor costo (analizar la relación costo-beneficio);
● Satisface criterios de performance y RNF;
● Cumple con expectativas de costo: incluye la satisfacción del equipo de proyecto.
No solo el cliente debe estar satisfecho, sino también el desarrollo del producto debe ser redituable.

Mientras el producto existe hay **actividad de gestión de configuración**, que es responsabilidad de todo el equipo. Sin embargo, existen roles específicos como el rol del Gestor de configuración que tiene tareas adicionales como por ejemplo mantener la herramienta, marcar línea base, etc.

¿Cuál es el problema que se está tratando de solucionar?

El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo.
Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento. Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión. Y todas las personas se enteren de los cambios.

Ítem de Configuración

Son artefactos que forman parte del producto/proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo.
Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida
(ya sea del producto o proyecto, dependiendo del artefacto).

Es cualquier aspecto asociado al producto de software que es necesario mantener
(requerimientos, diseño, código, datos de prueba, documentos, etc.)
Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario).

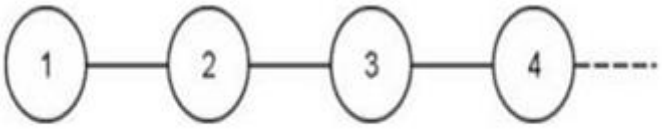
Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.
Algunos ejemplos: prototipo de interfaz, manual de usuario, ERS, arquitectura del software, casos de prueba, código fuente, íconos, etc, etc.

Versión

Instancia de un ítem de configuración que difiere de otras instancias de este ítem.
Las versiones se identifican unívocamente.
Controlar una versión refiere a la evolución de un único ítem de configuración.
"La gestión de configuración permite a un usuario especificar configuraciones alternativas del sistema de software mediante la selección de las versiones adecuadas";
Esto se puede gestionar asociando atributos a cada versión (pueden ser datos sencillos como un nro. de versión asociado a cada objeto)

Cada versión de software es una colección de elementos de configuración (ECS), como código fuente, documentos, datos. La versión es un punto particular en el tiempo de ese ítem de configuración.
Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
La evolución puede representarse gráficamente en forma de grafo.

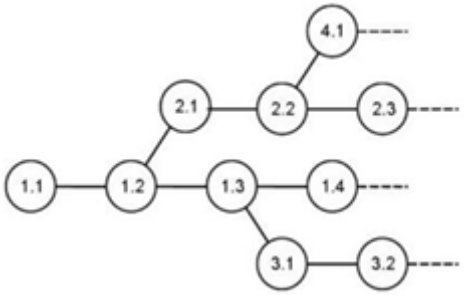
Control de versiones
Hace referencia a la evolución de un único ítem de configuración o de cada ítem de configuración por separado.



Evolución lineal de un ítem de configuración

Variante

Versión de un ítem de configuración (o de la configuración completa) que evoluciona por separado.
Las variantes representan configuraciones alternativas.
Las variantes de un ítem de configuración permiten satisfacer requerimientos en conflicto, como por ejemplo la compatibilidad del producto con diferente hardware o sistemas operativos.



Si la diferencia que existe entre las instancias de un mismo ítem de configuración es muy pequeña, no se denomina versión, sino que se la conoce como variante. Está ligado a trabajar con Branches, donde el producto adopta distintas configuraciones, dependiendo de múltiples factores (problemas, SO donde se ejecutará, etc.).

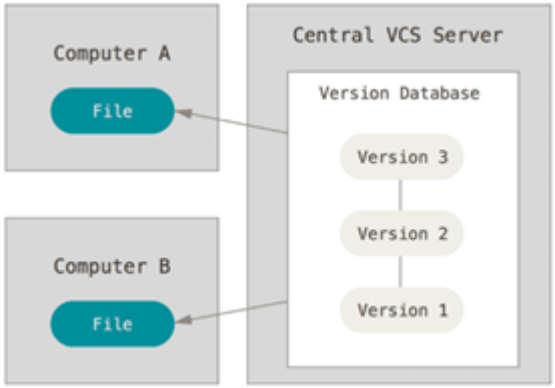
Configuración

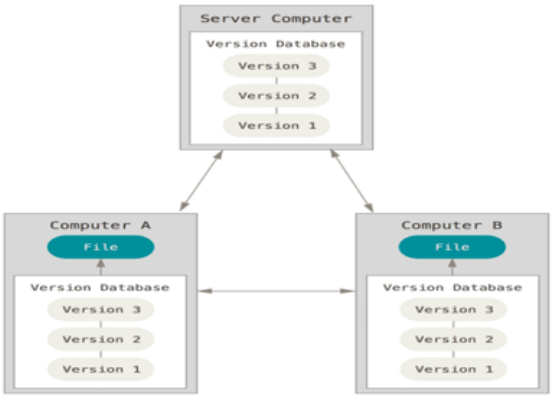
Conjunto de todos los ítems de configuración con su versión específica y la información de la estructura que definen una versión del producto a entregar.
Define una foto de los ítems de configuración en un momento de tiempo determinado.
La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

Repositorio

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos.
Puede ser una o varias bases de datos. Tiene una estructura para mantener el orden y la integridad.
El que tenga una estructura ayuda a la seguridad, controles de acceso, políticas de Backup y todo aquello que se aplica sobre un repositorio.
Tenemos 2 tipos de repositorios:
Repositorios Centralizados: Un servidor contiene todos los archivos con sus versiones.
Los administradores tienen un mayor control sobre el repositorio.
Tiene como desventaja qué si falla el servidor, todo lo positivo se cae.

Repositorios Descentralizados: Cada cliente tiene una copia exactamente igual del repositorio completo.
Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, posibilita otros workflows no disponibles en el modelo centralizado.
La desventaja es que es más complicado llevar un control sobre el mismo.





Release

Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización.

Rama (Branch)
Es un conjunto de ítems de configuración con sus correspondientes versiones, **que permiten bifurcar** el desarrollo de un software, por:
- Experimentación;
- Resolución de errores;
- Desarrollar un mismo software para diferentes plataformas.

Integración de ramas (Merge)
Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas.
Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella.
En caso de no integrarse a la Main, las ramas deberían descartarse.

Líneas Base Y Su Administración

Una línea base es un conjunto de ítems de configuración que han sido construidos y revisados formalmente. Los cuales pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores.
Esto es lo mismo que decir que es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.

Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.

Se acuerdan parámetros para determinar qué deben cumplir los ítems de configuración para considerarse como línea base.

Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios, y en caso de aceptarse, acordar nuevamente los parámetros y comunicar los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.

¿Para qué sirve una línea base?
Sirve como punto de referencia, pero también sirve para hacer Rollback o saber qué se pone en producción.
Nos permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando.

Existen distintos tipos de línea base:
Operacionales: contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente.
La primera línea base operacional corresponde con la primera Release.

De especificación / documentación: Son las primeras línea base, NO cuentan con ítems de configuración que sean código. Podría contener el documento de especificación de requerimiento.



Como los ítems de configuración no necesariamente tienen que ser codigo podemos ver que hay distintas líneas bases

Planificación de la gestión de configuración de software

Este plan se debe confeccionar al inicio de un proyecto, y existen diferentes estándares de cómo planificar:
- Reglas de nombrado para ítems de config
- Herramientas para usar en SCM
- Roles e integrantes del comité de control de cambios
- Procedimientos formales de cambios
- Estructuras de repositorios
- Tipos de documentos

Todo esto debe hacerse tempranamente, definirse los documentos y productos que se administrarán
Ejemplo:
"Todo código fuente, documento de requisitos aprobado, diseño de arquitectura, scripts de BD y documentación técnica serán gestionados como ítems de configuración."
Después, a medida que aparecen esos elementos, se incorporan siguiendo esas reglas.
Por lo tanto, se planifica tempranamente la gestión, no necesariamente se conoce de antemano cada archivo concreto.
Entonces con eso se refiere a productos (los cuales pueden ser o no ítems de config teniendo en cuenta como se defina)

Actividades relacionadas a la gestión de configuración

Elementos principales de SCM:
Informes de estado Control de cambios Identificación de ítems auditorias de configuración

Estas actividades/elementos son los que contienen las secciones de un plan de gestión de configuración.
Recordemos que la gestión de configuración también se debe planificar.
Todas las actividades se realizan en ambas metodologías, menos las auditorías, ya que el concepto de ser auditado y controlado por alguien externo, no es compatible con una metodología ágil pura.

Identificación de Ítems de Configuración

Implica una identificación unívoca para cada ítem.
En el equipo se definen políticas, reglas de nombrado y versionado.
Se debe definir tmb la estructura del repositorio, la ubicación de los ítems de configuración dentro de esa estructura y una definición cuidadosa de los componentes de la línea base.

Esta identificación y documentación proveen un camino que une todas las etapas del ciclo de vida del software, lo que permite a los desarrolladores controlar y velar por la integridad del producto, como así también a los clientes evaluar esa integridad.

Se debe tener en cuenta, el tiempo a conservar bajo control de integridad un item ya que no todos tienen la misma vida útil, **los podemos clasificar en:**
1. Ítems de producto:
Tienen el ciclo de vida más largo, y se mantienen mientras el producto exista.
Ejemplo: documento de arquitectura, casos de uso, código, manual de usuario, ERS, base de datos.
2. Ítems de proyecto:
Su ciclo de vida que abarca todo el proyecto, y se mantienen mientras el proyecto esté en ejecución.
Ejemplo: plan de proyecto, cronograma general, Burn-Down Chart.
3. Ítems de iteración:
Ciclo de vida más corto, limitado a una iteración, y se mantienen mientras esa iteración esté en desarrollo.
Ejemplo: plan de iteración, cronograma de iteración, reporte de defectos, Burn-Down Chart.

Luego de identificar los ítems se debe armar la estructura del repositorio con nombres representativos.
Luego se vinculan los ítems de configuración en el lugar físico del repositorio donde se van a almacenar.

Control de cambios

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Es decir, se hace un control sobre los ítems de configuración que pertenecen a la línea base (y se quieren cambiar), ya que todos los trabajadores del software tienen a dichos ítems como referencia.
El control de cambios permite tener una trazabilidad entre los ítems de configuración, y ante un cambio saber cuáles ítems están afectados por este.

Comité de control de cambios

Comité, el cual se reúne para autorizar la creación y cambios sobre la línea base.
Forman parte de dicho comité los interesados en evaluar y enterarse del cambio, decidiendo si lo aceptan o no. El líder del proyecto, analista, arquitecto e incluso el cliente pueden constituirlo.

Etapas del proceso

Si el comité fijó una línea base luego de la revisión de los ítems que la conforman y se concluye que los mismos se encuentran en estado estable, en caso de recibir una propuesta de cambio, los pasos son:

1. Se recibe una **propuesta de cambio sobre una línea base** determinada, **no sobre un ítem**.

2. El comité de control de cambios realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, efectos secundarios e impacto global sobre la funcionalidad y arquitectura del producto.

3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.

4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.

5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.

Auditorías de configuración

Esta actividad de la disciplina SCM no es soportada por la filosofía agil

Son controles autorizados por el equipo de desarrollo, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten congelar y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.
- Para que sea objetiva el auditor no debe estar condicionado con respecto a lo auditado
- Es independiente cuando el auditor no participa de la jerarquía, del equipo a auditar
(básicamente que el auditor no sea del equipo / área / cobre sueldo por el equipo a auditar)

La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad
La auditoría de configuración complementa a la revisión técnica y consiste en revisar si se están realizando las tareas tales como se planificaron y especificaron en el plan de gestión de configuración. Las auditorías se realizan sobre una línea base. Es decir, la auditoría requiere la existencia de un plan de gestión de configuración y de la línea base a auditar.

Control de cambios = decidir y autorizar cambios.
"¿Podemos cambiar esto?"

Auditoría = comprobar que la configuración cumple lo establecido.
¿Está todo correctamente después de los cambios?"

	Control de cambios	Auditoría de configuración
Objetivo	Decidir y controlar **qué cambios se pueden hacer**	Verificar **si se hizo todo correctamente**
Momento	Antes y durante la implementación del cambio	Después, para verificar el estado
Quién participa	Comité de Control de Cambios	Auditor independiente
Pregunta principal	"¿Autorizamos este cambio?"	"¿La configuración cumple las reglas?"
Analiza	Impacto, esfuerzo, recursos, riesgos	Cumplimiento, integridad y consistencia
Resultado	Orden de cambio / autorización	Informe de auditoría

La auditoría le sirve a:
1. Validación: asegura que cada ítem de config resuelva el problema apropiado, lo que el cliente necesita.
2. Verificación: asegura que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status "ok" o bien consten como "problemas reportados" en la nota de reléase.

Tipos de auditorías

1. Auditoría de contingencia: se realiza primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración
• Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo.
• Lo que se audita es una línea base.
2. Auditoría funcional de configuración: audita la trazabilidad de requerimientos, controla que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos. El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS. Antes de comenzar, pregunta si está disponible el reporte de la auditoría de contingencia. Ya que, si la auditoría física no sale bien, la auditoría funcional no se hace.

Trazabilidad o rastreabilidad de requerimientos

La trazabilidad de requerimientos consiste en poder seguir un requerimiento a lo largo de todo el desarrollo del software y saber qué elementos están relacionados con él.
Si tenemos un requerimiento A, durante su desarrollo va a generar distintos elementos como
Análisis – diseño – Codigo – pruebas
y la trazabilidad permite partir de este Req A y descubrir que cosas podrían verse afectadas
Req A cambia → análisis → diseño → código → pruebas
Entonces sé qué documentación revisar, qué componentes modificar y qué pruebas posiblemente actualizar.
Sin trazabilidad, tendría que investigar manualmente qué partes del sistema dependen de ese requerimiento.

¿Qué significa bidireccional?

Que podemos recorrer la relación **en ambos sentidos**:
Requerimiento → Código:
"¿Qué código implementa este requerimiento?"
Código → Requerimiento:
"¿Qué requerimiento justifica o corresponde a este código?

Informe de estado

Provee un mecanismo para mantener un registro de cómo evoluciona el sistema.
Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.
Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Normalmente consta de grandes unidades de datos, por lo que se usan herramientas automatizadas por una computadora.
El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados.
Permite responde a preguntas como:
a. ¿Cuál es el estado del ítem?
b. ¿La propuesta de cambio fue aceptada o rechazada por el comité?
c. ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
d. ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

gestión de configuración en ambientes ágiles

En Agile la gestión de configuración está pensada para **ayudar al equipo de desarrollo**, y no para que el equipo tenga que trabajar principalmente para satisfacer las necesidades administrativas de SCM.
La gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.
Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. Todos los procesos definidos establecidos en la gestión de configuración del enfoque tradicional son relativizados en agile. Por ejemplo, no existe un comité para el proceso forma de control de cambios.
La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto.
Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.

Se trabaja con integración continua (CI), un servidor donde se testea automáticamente el código y luego se integra en producción continuamente.
Un repositorio, un esquema de nombrado son cosas que, si se utiliza en ágiles, mientras que la auditoría y el comité de control de cambios durante el sprint, pueden obviarse.

Continuous Integration

Práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día.
Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. integrar frecuentemente resulta más fácil detectar y corregir los errores.

La integración continua es asegurar que el software pueda ser desplegado en cualquier momento, es decir, que el código compile y que sea de calidad.

Cada desarrollador en su entorno de trabajo realiza pruebas unitarias (en la medida de lo posible, automatizadas) desarrollando con algo que se llama TDD (desarrollo conducido por pruebas) y cuando terminó ese componente de código y lo probó y sabe que funciona, lo sube a un repositorio de integración. Así, la versión del producto está en condiciones de ir a las pruebas de aceptación de usuario sin problemas

Continuous Delivery

Disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento.
Suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.
No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, PO de turno, decide pasar a producción. Es decir que hay un componente «humano» a la hora de tomar la decisión. Pero, en cualquier caso, la versión está lista de inmediato.
Esto implica, que se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.
La integración continua es prerrequisito de la entrega continua.
Hay que asegurarse de tener un despliegue automatizado, para que cada puesta en producción no lleve demasiado tiempo, lo que hará que las puestas puedan llegar a hacerse más seguidas, generando que lo que se despliegue no sean demasiados cambios y el riesgo de que algo se rompa disminuya.
El software debe estar siempre en un estado de "entregable", es decir, el software buildea, el código se compila y los tests pasan

Continuous Deployment

Consiste en poner en el ambiente de producción del usuario final el producto.
A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.
El propósito es que sea transparente para el usuario que pusiste en producción una nueva versión del producto. Se recomienda hacer el despliegue a poco tiempo de haber trabajado con los cambios en el código, pues un error en producción un día después de haberlo hecho significa que todavía nos acordamos de lo que hicimos y será más fácil de resolver.

Canary Deployment

Es la práctica de realizar lanzamientos por etapas.
Primero implementamos una actualización de software para una pequeña parte de los usuarios, para que puedan probarla y proporcionar comentarios.
Una vez aceptado el cambio, la actualización se extiende al resto de usuarios. Esta estrategia nos muestra cómo los usuarios interactúan con los cambios de la aplicación en el mundo real.
Ofrece actualizaciones sin tiempo de inactividad y reversiones sencillas.
Las implementaciones son fluidas y las fallas tienen un impacto limitado.

Blue-Green Deployment

Es un modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación a una nueva versión casi idéntica, las cuales se ejecutan en producción.
La versión anterior puede denominarse entorno azul, mientras que la nueva versión puede denominarse entorno verde. Una vez que el tráfico de producción se transfiere por completo de azul a verde, el azul puede quedar en espera en caso de reversión o retirada de producción y se actualiza para convertirse en la plantilla sobre la que se realiza la próxima actualización.

Integración continua (CI)

Consiste en que los desarrolladores integran frecuentemente sus cambios al código principal y automáticamente se ejecutan controles.
Continuous Delivery (Entrega continua)
Va un paso más allá: cada cambio que pasa los controles queda preparado para ser entregado/desplegado.
Continuous Deployment:
Si pasa todos los controles, además se despliega automáticamente a producción.